UNIT DETAILS:

Unit Code	COS30049
Unit Name	Computing Technology Innovation Project
Lecturer	Mohamad Faizal Alias
Assignment	Assignment 3: SchoolAir Technical Report
Section	C1
Session	August 2026
Presentation Link	https://youtu.be/aqgMAgEUqZM

STUDENT DETAILS:

Group 7			
Student Name	**Student ID**	**Position**	**Signature**
Poh Wei Le	106627706	Leader	*WeiLe*
Ng Boon Ping	106627829	Member	*BoonPing*
Kennard Chan Kai Lik	106632645	Member	*Kennard*
Tee Ming Jun	106627816	Member	*MingJun*

Table of Contents

Executive Summary 4

1.0 Introduction and Project Scope 5

1.1 Background and Decision Problem 5

1.2 Objectives 6

2.0 System Architecture 7

2.1 Component Boundaries 7

2.2 Responsibilities and Interfaces 8

2.3 Request Lifecycle and Two Prediction Paths 9

2.4 Verified Example 10

2.5 Data Assets and Storage 11

2.6 Operational Trade-Offs 12

3.0 Frontend Implementation 13

3.1 Navigation, Inputs and Shared Selection 13

3.2 Reading the Result 14

3.3 Event Assessment Visualizations 15

3.4 Event Averages and Hourly Values 16

3.5 Explaining the Prediction 17

3.6 User Feedback During Changes 18

3.7 Seasonal Planner and Comparison 19

3.8 Schedule Workflow 20

3.9 Analytics Charts and Responsive Design 21

4.0 Backend Implementation 23

4.1 FastAPI Service and Validation Boundaries 23

4.2 Development Configuration 24

4.3 Event Storage and Update Behaviour 25

4.4 Deployment Limitations 26

5.0 API Documentation **27**

5.1 Request Conventions and Endpoint Inventory 27

5.2 GET Forecast Lookup 28

5.3 POST Model Prediction 30

5.4 GET Seasonal Planner 31

5.5 GET District Comparison 32

5.6 Event Create, Read, Update, and Delete 33

5.7 GET Analytics and Station Context 34

5.8 API Coverage and Conventions 35

6.0 Model Integration **36**

6.1 Model Selection From Assignment 2 36

6.2 Preparing Historical Profiles For Serving 37

6.3 Feature Schema 38

6.4 Loading The Model and Executing A Request 39

6.5 Postprocessing, Recommendations and Explainability 40

7.0 Testing and Evaluation **41**

7.1 Verification Approach 41

7.2 UI interactions and Responsiveness 43

7.3 CSV and Print Verification 44

7.4 Evidence For Advanced Functionality 46

7.5 Assignment 1 & 2 Risk Resolution and Final Handover Matrix 47

8.0 Conclusion and Future Work **48**

8.1 Project Outcomes 48

8.2 Priorities For Future Development 48

8.3 Expanding Coverage In Peninsular Malaysia 49

9.0 Bibliography **50**

10.0 Appendices **51**

Appendix A: Client Assessment Request (App.js) 51

Appendix B: Assessment and Calendar Validation (main.py) 53

Appendix C: Forecast Lookup Assessment Route (main.py) 55

Appendix D: Window Selection and Aggregation Logic (main.py) 56

Appendix E: On-Demand Model Prediction Route (main.py) 57

Appendix F: Seasonal Planner Response Builder (main.py) 59

Appendix G: D3.js Historical Trend Brush Implementation (DiurnalTrendD3.js) 61

Appendix H: Browser Client-Side CSV Export Utility (exportUtils.js) 63

Appendix I: MOE Safety Directive Classification Engine (main.py) 64

Appendix J: Scheduled Event Persistent Update Handler (main.py) 66

Appendix K: Automated Model Regression Test Suite (test_api.py) 68

Appendix L: Meeting Minutes 69

Appendix M: Group Assessment Contribution Form 79

Executive Summary

SchoolAir is full-stack planning software for outdoor events at Malaysian schools, using a React interface and the FastAPI service, with the XGBoost regressor chosen in Assignment 2. It provides a user with different estimates of air quality during the event, the ability to compare months and regions, and to check the information visually. The data includes 26 different monitoring stations, while the states of Kelantan, Terengganu, and Pahang are not included in this release.

For its usual evaluation, the Safety Checker utilizes historical estimates by season. Nevertheless, an additional option is available, which allows the model to be executed on the server with the use of historical feature rows. Thus, both outputs are distinctly separated from each other. For example, regarding Petaling Jaya on April 15, 2026, from 8 to 10 am, one output is 36.05 API obtained through the lookup, while another one is 36.0 API received from the model version.

The Planner employs monthly summaries for comparing seasons while keeping processed data in scheduled events that were either created, updated, or deleted, using different routes. Analytics provides a regional heatmap, and the historical haze incident is brushable. The application supports chart types such as dial gauges, line charts, horizontal feature attribution charts, and heatmaps. The outputs in CSV or printable formats keep the event context so that necessary conclusions can be reached outside the web browser.

The verification process covered the production build of the application front end, nine automated tests, fluid browser interactions, export checks, and the stand-alone event lifecycle. The resulting solution enables advanced planning. SchoolAir uses historical seasonal estimates for planning, and schools should check current air-quality observations and applicable guidance on the day of the event. Further evaluation must assess seasonal estimation error and include sufficient observations in high-risk situations to estimate proper performance in rare conditions.

1.0 Introduction and Project Scope

1.1 Background and Decision Problem

The planning of a school's sport day entails decisions on time and place, as well as the nature of the activity itself. In cases where air quality is a factor, it is necessary for the planning committee to be aware of the differences in air quality among the different timings and places. This is what SchoolAir does. It takes the comparison of air quality among different times and places into one application. This application utilizes the Air Pollutant Index or API in short form.

Assignment 1 identified school administrators as the primary users, with physical education teachers and sports coordinators as related users (Group 7, 2026a). The requirements in assignment 1 relate the requirement for comprehensible recommendation to the choice of a station, risk classification, and visibility graphs. In Assignment 2, predictions were compared and the model chosen for the continuous index is XGBoost regressor. Assignment 3 implements the requirements and modelling outcomes from the previous assignments within a browser workflow, backed by a backend service.

The final system has the functionality for planning at three scales. Safety Checker assesses a chosen period for an event. Event Planner assists in comparing the months and districts as well as maintaining a schedule. Analytics allows for exploration of a broader station-by-month pattern and haze event from the archive.

1.2 Objectives

There are four goals of the implementation which include provision of clear estimates for an event selection, presentation of Assignment 2 model through server-side inference, variation explanation through interactive D3 graphs, and provision of practical follow-up through comparison, scheduling and exports. The application employs historical seasonal profiles. This is a limited release in terms of coverage because only the 26 stations from the Assignment 2 dataset are covered. The packaged data does not contain any station in Kelantan, Terengganu or Pahang.

Table 1.1 Prior Project Requirements and Application Evidence

Earlier project need	**Delivered feature**	**Evidence in this report**
Interpret risk quickly	Safety Checker estimate and tier	Figures 2.1 - 2.2
Choose a suitable planning period	Seasonal Planner and comparison	Figures 2.5 - 2.6
Understand variation and model drivers	Hourly curve, SHAP and Analytics	Figures 2.2 - 2.4 and 2.7 - 2.8

2.0 System Architecture

2.1 Component Boundaries

With SchoolAir, the functionalities namely, browser presentations, logic of application at server, and asset storage have been segregated. In the browser an input from a user is captured and presented as an output. FastAPI process requests and obtains the necessary information, calculates and returns the data to be presented as JSON. The backend component of the system performs storage of model files, feature schema and historical datasets. Thus, the browser requires returned values and labels to conduct rendering of the application while the backend keeps the underlying logic informing meanings for it.

The architecture of the system claims the usage of React to establish component architecture for web pages and reusing components. Material UI provides user interface design components for rendering. Axios establishes connections between client-server and acts as an interface between HTTP request and page states. D3 forms the visual part of the charts and initializes interactions with it. In conclusion, the system architecture is that the district or time is changed, and the relevant display is refreshed accordingly without reloading the application.

The FastAPI framework provides the functionality of requesting handling in Python, along with the interactive OpenAPI documentation. To keep the server in Python means to use similar data processing algorithms and libraries such as XGBoost in the modeling processes. The implementation utilizes folders and files relative to the position of the backend module in order to load pre-trained models and data resources. The front end provides a developer-configured API base URL, which allows to separate the location of the web app from the development server.

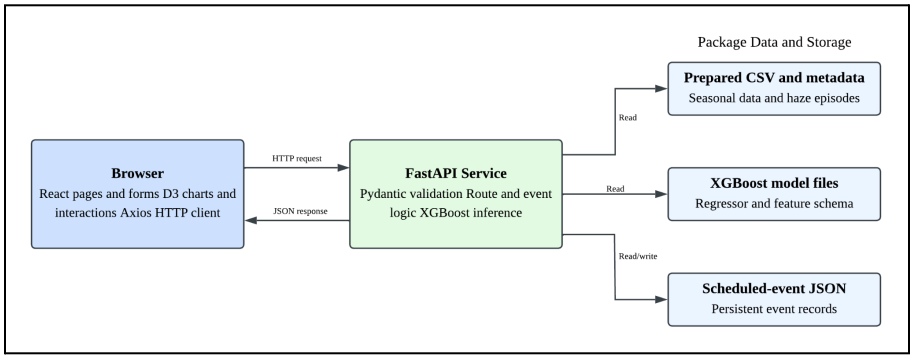


Figure 1.1 SchoolAir Component Architecture

2.2 Responsibilities and Interfaces

The frontend manages the interface, navigation, filters, and feedback shown to users. The server handles station search, the time interval calculations, and the suggestions given with the estimate. The backend stores scheduled events and returns them when the Planner requests the saved list. Exporting features are implemented on the front end since the pertinent report data exists in the evaluation or planning response.

2.3 Request Lifecycle and Two Prediction Paths

The common assessment can be obtained through the GET /api/v1/forecast-lookup request. React sends the event details chosen to FastAPI, which creates a valid request, identifies the station and the month, and performs filtering on the available hourly forecasts. It uses those hours to compute the summary statistics and returns them with the weather information, the recommendations, the hourly series, and the feature contributions from the database. These pieces of information serve as the backbone of several different views of one and the same assessment.

The Advanced model tools functionality uses the POST /api/v1/model-prediction request. The backend uses the same dataset to identify the station, the month, and the event hours and to build the input matrix from the available seasonal characteristics and to execute the regressor. The response includes the newly computed index as well as the historical baseline for comparison. This comfortably allows users to compare the model run with the assessment that has already been visualized in the charts.

The two processes work on different operational principles. The lookup has a pre-prepared table while the prediction path requires the regressor to be loaded into memory and will return an HTTP 503 error in case it is not loaded. Therefore, it is possible to provide error information about a failed inference while the lookup was successful. Input provenance is the same limiting factor in both processes, as both use historical seasonal data for future planning.

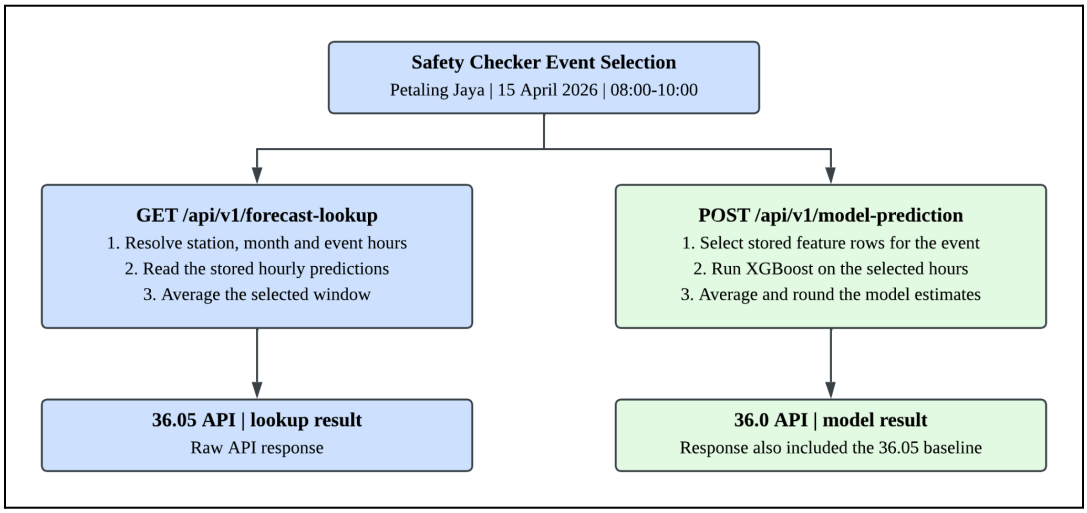


Figure 1.2 Assessment Lookup and On-Demand Inference

2.4 Verified Example

A request for Petaling Jaya for April 15, 2026, for 08:00–10:00 produced 36.05 in case of the lookup and 36.0 in case of the on-demand run. The example used shows that in the examined case the lookup value is rounded to 36.0.

2.5 Data Assets and Storage

The backend loads station and school directories, hourly forecasts, monthly summaries, historical haze episodes, and analytics metadata. It also stores scheduled events in a JSON file. Table 2.1 lists each asset and its role.

Table 2.1 SchoolAir Data Assets and Roles

Asset	Logical key or scope	Use
clean_locations.csv	District or monitoring station	Directory, corridor filters, station resolution
schools_directory.csv	School directory entry	Named school and matched station metadata
precomputed_hourly_forecasts.csv	District, month, hour	24-hour chart, event window, features and contributions
precomputed_monthly_summary.csv	District and month	Planner, rankings and regional heatmap
precomputed_multiday_episodes.csv	District and timestamp	Archived haze comparison chart
analytics_metadata.json	Model and corridor summaries	Benchmark and correlation metadata returned by Analytics
scheduled_events.json	Event identifier	Server-side schedule persistence

The hourly forecast file contains 7,488 rows covering 26 stations, 12 months, and 24 hours per station-month. The monthly summary contains 312 records, one for each station and month, and supports the Planner and regional heatmap. The dataset for historical episodes is based on timestamps with observations and model series. The station list and all three prepared datasets relate to the exact same 26 stations.

2.6 Operational Trade-Offs

One of the benefits of preloaded tables is that it enables the service to choose and collect rows without re-training the model or contacting the data supplier for each request. The JSON schedule is easy to analyze and excellent for a local demonstration. However, its read-modifying-writing procedure does not provide database transactions and guarantees of concurrent writing. These deficiencies must be addressed before starting to use the service on the school-wide level.

3.0 Frontend Implementation

3.1 Navigation, Inputs and Shared Selection

The Home page allows access to the Checker, Planner, and Analytics. Their selectors reduce the scope before a user launches Safety Checker. The React state contains the location and event selections that are needed in other views. The state filter represents the available stations directory. A preset school name fills in the corresponding location. Entering a new school name gives the report for the chosen station but does not add a forecast scope.

Safety Checker gets the region or state, district, school, date of event, hour, and activity. The form presents the date in day/month/year format and transforms a proper date into the ISO format. Date validation on the calendar excludes impossible dates instead of accepting any date-looking data. The server performs independent verification of the date and numeric data received.

The Checker organizes the form, recommendation, summary cards and graphs all based on the same selection. Thus, the user does not have to construct his/her own assessment based on the information from several different pages. The interface also determines if there is consistency between the existing assessment and the new one corresponding to the current input combination.

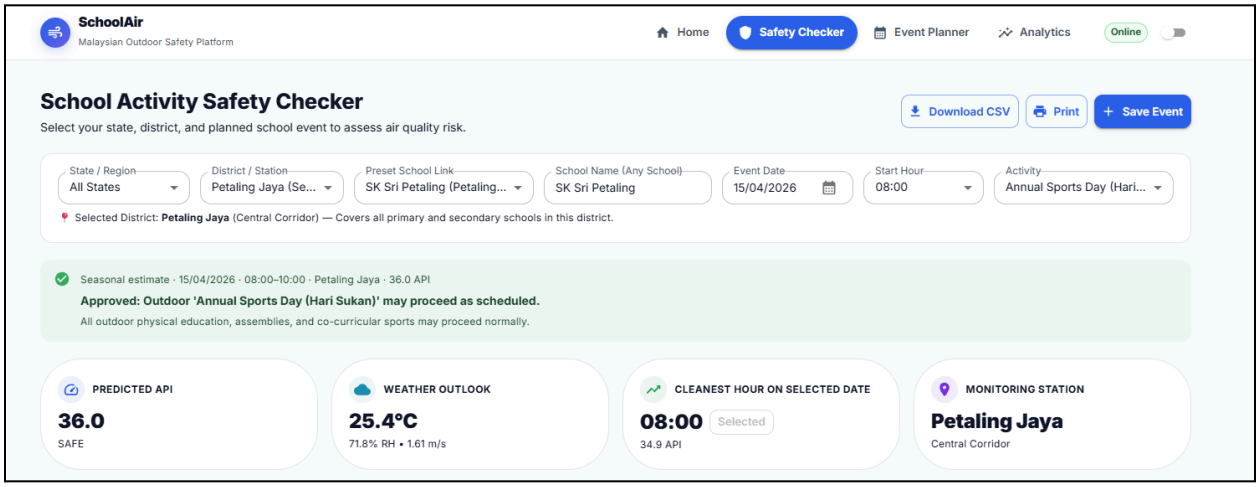


Figure 2.1 Safety Checker Inputs and Result Summary

3.2 Reading the Result

Recommendation turns the numeric value into a specific plan. The cards give supporting information: the forecasted index, weather conditions, the cleanest suitable daylight hour, and the monitor location. The name of the station indicates the observation environment where the estimate was made. The selected time range is especially critical when saving/exporting a report as the other start hour will affect the event average.

3.3 Event Assessment Visualizations

In the Safety Dial Gauge, D3 is used for encoding the average event window as the position of the pointer on a colored dial. The numeric value and tier label are independent of the color. Therefore, the output can be usable even when the risk bands are not identifiable based on color. The hover information describes the interval captured by each zone in the gauge. The responsive size keeps the dial small on smaller screen sizes without hiding the threshold labels.

The 24-hour graph shows the information which is not conveyed through the gauge. It shows how estimated air quality changes over the day and highlights the selected window, the clearest hour, and the peak hour. Point selection in the graph allows changing the event starting point. The point readout helps users in analyzing the value for a particular hour before selecting it. The temperature and haze overlay selectors allow changing the secondary series.

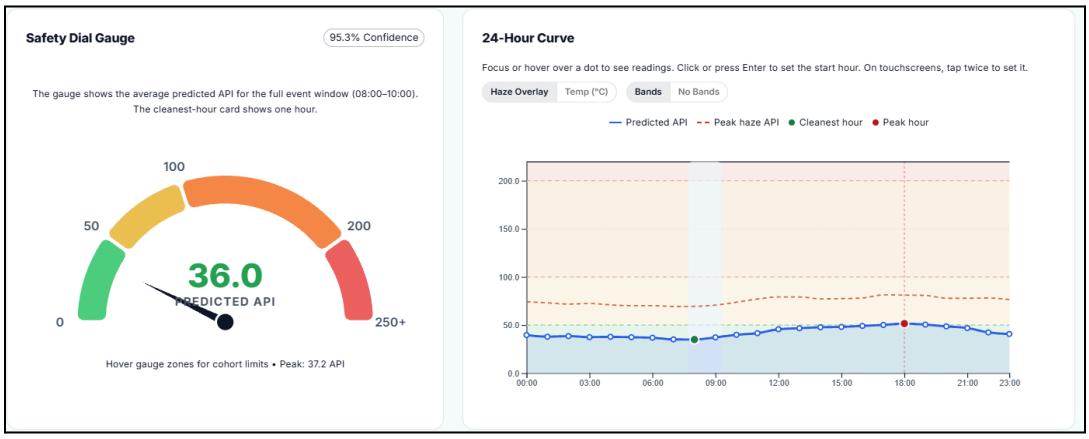


Figure 2.2 Safety Dial and 24-hour API Curve



Figure 2.3 Temperature Overlay and Highest Temperature Marker

3.4 Event Averages and Hourly Values

The event average is calculated for all the hours in the window chosen. The cleanest-hour card provides the least reading out of all the eligible daytime hours. In the above example, the reading on the cleanest hour is 34.9 at 08:00, whereas that in the event-window is 36.0. Both readings are relevant since each provides information for its purpose.

3.5 Explaining the Prediction

The Advanced model tools section includes the optional model run and the Feature Attribution chart. The primary assessment is still available as a separate request runs. On receipt of the response, the panel recognizes that it is a result of the recomputation of the previously saved historical inputs. This makes it possible for the user to examine the model's performance without interrupting the lookup-based assessment.

In the SHAP visualization, horizontal bars are used to show what input variables were the most important for the selected hourly outcome. There is an option of viewing the relative share and the signed index contribution. The relative share shows the relative importance, while the signed index tells the user whether the contribution is making the model output higher or lower than the baseline value. The chart uses saved contributions related to the selected start hour of the assessment.

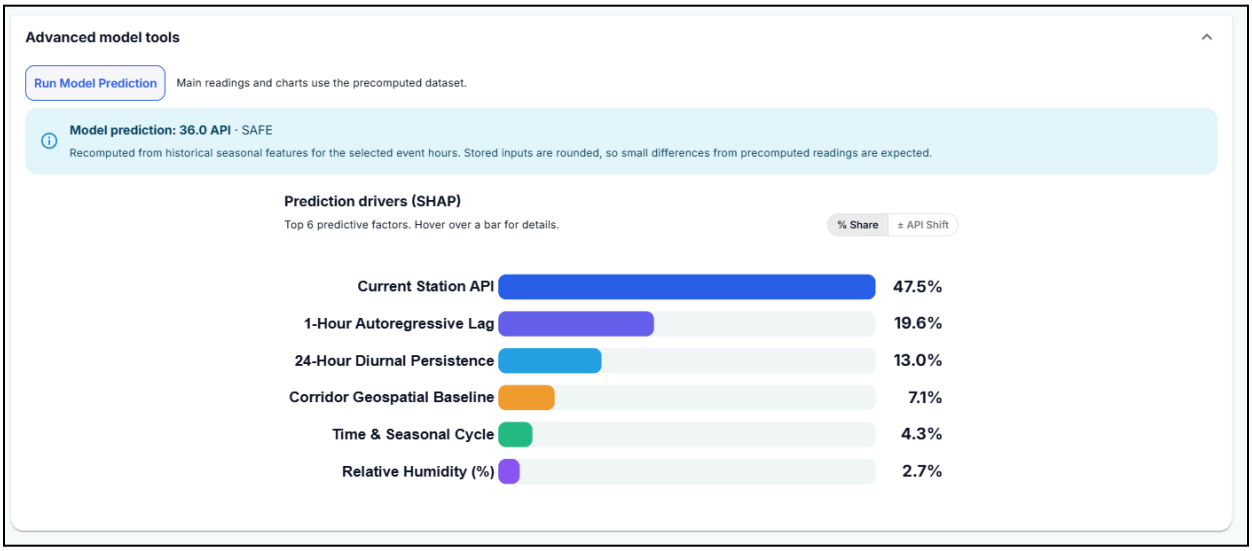


Figure 2.4 Advanced Model Tools and SHAP Feature Attributions

3.6 User Feedback During Changes

The model result is related to the selection it has been produced from. If the user changes the date, station, or hour, then the frontend removes the old model result. The request counter will ensure that the outdated answer will not be displayed in connection with the new selection. In a similar way, the design is applied to the regular assessment: the loading status and input key help to recognize if the visible result is connected to the current form.

With the help of using different panels, the identification of mistakes will become easier. A model mistake is related to the optional prediction process, while the lookup mistake concerns the assessment itself.

3.7 Seasonal Planner and Comparison

The Planner shows a 12-month analysis for the chosen station. Users can view the seasonal risk designation, pick a favourable month, and select a month on the Safety Checker. The year shown here serves as a calendar basis for planning. The Planner reuses the same historical monthly profiles across planning years. The selected year sets the calendar context for the event.

Comparison between schools employs a common month and start hour in analyzing two districts. These two serve as constants, while location becomes the varying factor. In the April 08:00 example that was validated, the system found that Shah Alam's estimated API was 0.6 points lower than Petaling Jaya's for the selected April event window. The number constitutes a slight deviation in the model. Therefore, it allows comparison without proving any actual difference on the given date.

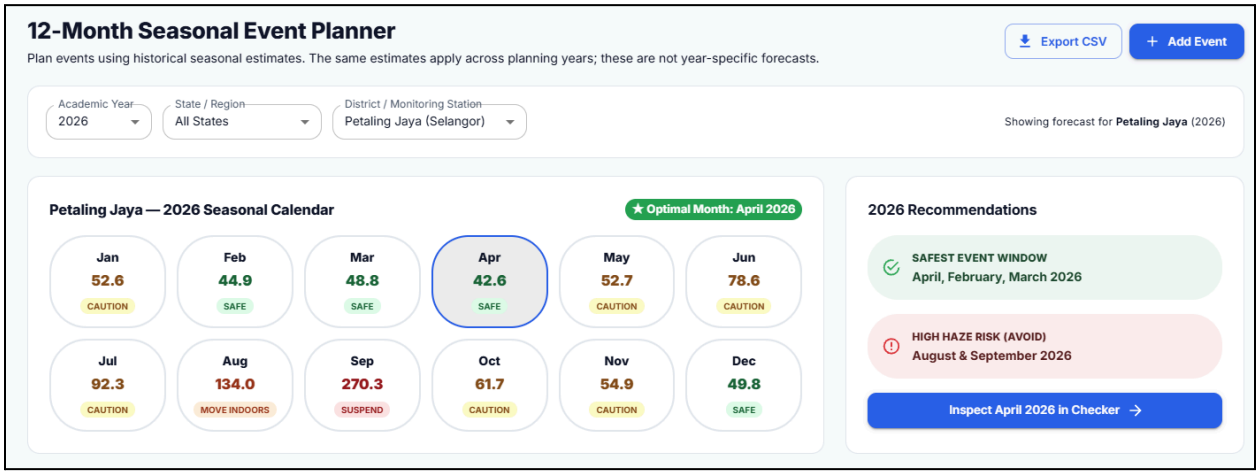


Figure 2.5 Seasonal Planner Calendar and Recommended Months

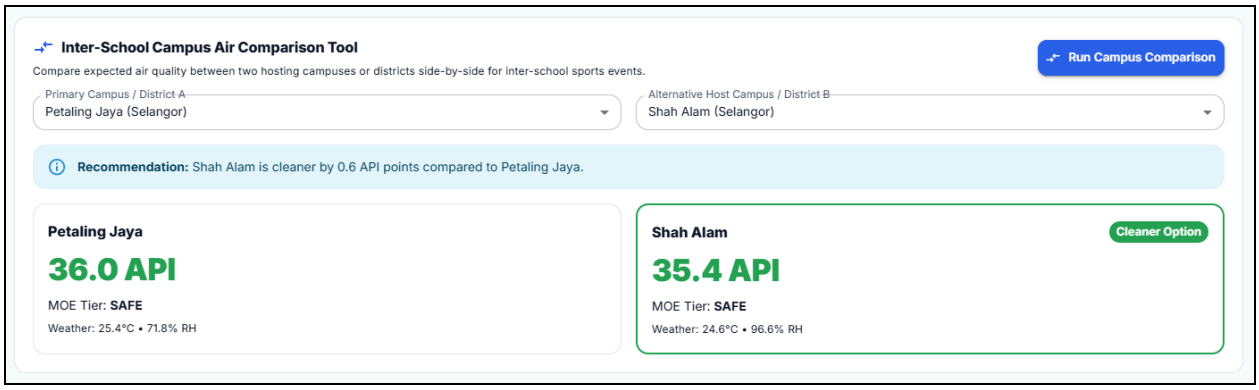


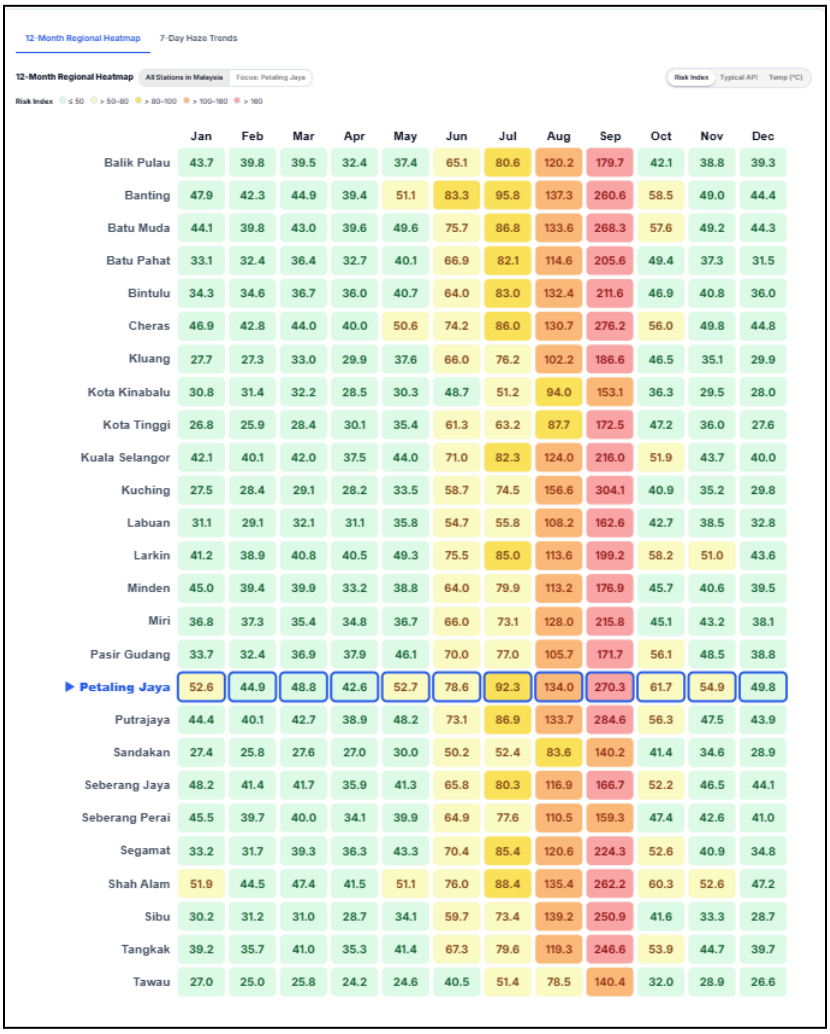
Figure 2.6 Campus Air Comparison Results

3.8 Schedule Workflow

An event can either be added by a user in the Planner or a saved assessment in the Safety Checker can be done. The schedule displays the saved data and allows updates and deletion where applicable. In the Planner, users can move a saved event to the recommended hour with the lowest estimated API.

3.9 Analytics Charts and Responsive Design

Environmental analytics offers macro-environmental insights through two optimized D3.js components: 12-Months Regional Heatmap and 7-Days Haze Trends. The 12-Month Heatmap component links to the parent station filter such that a user can switch between the single station view (Focus: <Station>) and the corridor-wide view (All Stations in <Corridor>) in which the currently selected station is represented by ▸ and the cell borders colored blue (#2563eb). The redundant components (correlation matrix and gauge/SHAP charts) have been removed in order not to overcrowd the dashboard with unnecessary views and focus only on seasonal trends. In the 7-Days Haze component, d3.brushX allows a user to zoom in and analyze both the actual API readings and the XGBoost predictions during the September 2019 haze episode with MOE alerts filling to the ceiling without top-margin space.



12-Month Regional Heatmap | 7-Day Haze Trends

12-Month Regional Heatmap | All Stations in Malaysia | Focus: Petaling Jaya | Risk Index | Typical API | Temp (°C)

Risk Index: ≤ 50 | > 50–80 | > 80–100 | > 100–160 | > 160

	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Balik Pulau	43.7	39.8	39.5	32.4	37.4	65.1	80.6	120.2	179.7	42.1	38.8	39.3
Banting	47.9	42.3	44.9	39.4	51.1	83.3	95.8	137.3	260.6	58.5	49.0	44.4
Batu Muda	44.1	39.8	43.0	39.6	49.6	75.7	86.8	133.6	268.3	57.6	49.2	44.3
Batu Pahat	33.1	32.4	36.4	32.7	40.1	66.9	82.1	114.6	205.6	49.4	37.3	31.5
Bintulu	34.3	34.6	36.7	36.0	40.7	64.0	83.0	132.4	211.6	46.9	40.8	36.0
Cheras	46.9	42.8	44.0	40.0	50.6	74.2	86.0	130.7	276.2	56.0	49.8	44.8
Kluang	27.7	27.3	33.0	29.9	37.6	66.0	76.2	102.2	186.6	46.5	35.1	29.9
Kota Kinabalu	30.8	31.4	32.2	28.5	30.3	48.7	51.2	94.0	153.1	36.3	29.5	28.0
Kota Tinggi	26.8	25.9	28.4	30.1	35.4	61.3	63.2	87.7	172.5	47.2	36.0	27.6
Kuala Selangor	42.1	40.1	42.0	37.5	44.0	71.0	82.3	124.0	216.0	51.9	43.7	40.0
Kuching	27.5	28.4	29.1	28.2	33.5	58.7	74.5	156.6	304.1	40.9	35.2	29.8
Labuan	31.1	29.1	32.1	31.1	35.8	54.7	55.8	108.2	162.6	42.7	38.5	32.8
Larkin	41.2	38.9	40.8	40.5	49.3	75.5	85.0	113.6	199.2	58.2	51.0	43.6
Minden	45.0	39.4	39.9	33.2	38.8	64.0	79.9	113.2	176.9	45.7	40.6	39.5
Miri	36.8	37.3	35.4	34.8	36.7	66.0	73.1	128.0	215.8	45.1	43.2	38.1
Pasir Gudang	33.7	32.4	36.9	37.9	46.1	70.0	77.0	105.7	171.7	56.1	48.5	38.8
▸ Petaling Jaya	52.6	44.9	48.8	42.6	52.7	78.6	92.3	134.0	270.3	61.7	54.9	49.8
Putrajaya	44.4	40.1	42.7	38.9	48.2	73.1	86.9	133.7	284.6	56.3	47.5	43.9
Sandakan	27.4	25.8	27.6	27.0	30.0	50.2	52.4	83.6	140.2	41.4	34.6	28.9
Seberang Jaya	48.2	41.4	41.7	35.9	41.3	65.8	80.3	116.9	166.7	52.2	46.5	44.1
Seberang Perai	45.5	39.7	40.0	34.1	39.9	64.9	77.6	110.5	159.3	47.4	42.6	41.0
Segamat	33.2	31.7	39.3	36.3	43.3	70.4	85.4	120.6	224.3	52.6	40.9	34.8
Shah Alam	51.9	44.5	47.4	41.5	51.1	76.0	88.4	135.4	262.2	60.3	52.6	47.2
Sibu	30.2	31.2	31.0	28.7	34.1	59.7	73.4	139.2	250.9	41.6	33.3	28.7
Tangkak	39.2	35.7	41.0	35.3	41.4	67.3	79.6	119.3	246.6	53.9	44.7	39.7
Tawau	27.0	25.0	25.8	24.2	24.6	40.5	51.4	78.5	140.4	32.0	28.9	26.6

Figure 2.7 Regional Seasonal Heatmap and Cell Detail

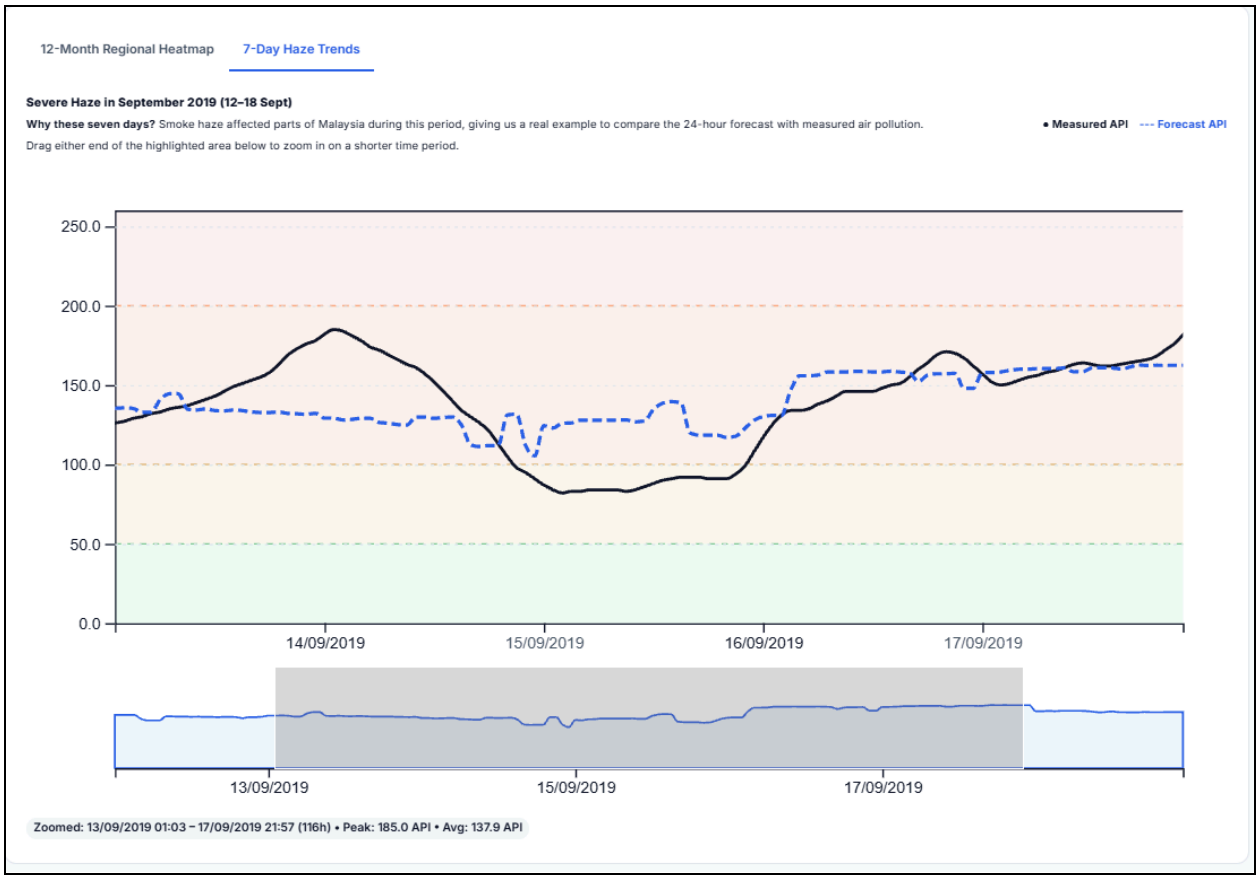


Figure 2.8 Historical Haze Trend and Brush Selection

4.0 Backend Implementation

4.1 FastAPI Service and Validation Boundaries

The backend organizes all the routes under path /api/v1. A universal app instance provides routes management, error responses, and cross-origin resource sharing. Interactive docs of FastAPI facilitate the usage of various methods, schemes, and parameters during development. Successful handlers return structured JSON so the frontend can render specific fields without parsing prose.

Pydantic models are used by the service for assessment and scheduling inputs. Assessment start hours can be in the range of 0 to 23 and the duration can be from 1 hour to 12 hours. The scheduling request defines other limits: the start hour can be between 6 hours and 20 hours and the duration can be anywhere from 1 to 10 hours. The number of participants can also vary in the range 1-5,000. Calendar validation checks that dates actually exist. These rules correspond to the general assessment tool capabilities and the specific event scheduling process.

Table 3.1 Validation Responsibilities Across The Application

Check	**Browser responsibility**	**Server responsibility**
Calendar date	Parse and show an input error for an impossible date	Validate the ISO value and reject malformed or impossible dates
Hour and duration	Offer supported choices in the UI	Apply numerical bounds to assessment and scheduling separately
Model scenario	Send the standard historical model selection	Reject unsupported overrides on the dedicated model route
Missing resource	Display response error or unavailable state	Return the handler’s error status and message where applicable

The assessment route has a specific fallback worth documenting. In the district-based branch, an unrecognized district resolves to Petaling Jaya. The Planner instead returns HTTP 404 for an unknown district. If an unsupported district is sent directly to the Checker API, a response for Petaling Jaya does not represent local conditions in the requested district. A future coverage check should make unsupported requests explicit.

4.2 Development Configuration

The checked service allows broad cross-origin access for development. This helps a local React server call the backend on another port. A shared deployment would need explicit origin restrictions, identity, access controls and durable storage.

4.3 Event Storage and Update Behaviour

A scheduled event stores an identifier, school and location information, timing, and a result calculated at save time. Creation checks the request, performs the assessment, and saves the record. The returned event lets the system insert the saved row without needing to retrieve the whole list. The next request for the list retrieves the saved records and can filter on the basis of the corridor.

To update a record, an identifier has to be sent through the path and a JSON body has to include supported fields. The server processes the update and performs the assessment recalculation based on the valid event's information. A changed start hour can therefore change the stored index and tier. The existing location remains part of that assessment. A different deletion path removes the matching identifier and gives a response back about the deleted identifier.

In an isolated temporary schedule file, a verification run was performed. The create request received an HTTP 201 code, the list of saved data was returned, and an update made at 09:00 resulted in a change in predicted index from 36.05 to 38.53 and a deletion that provided the same event identifier. The test checked persistence logic without writing the record to a regular event store. The update and delete calls were issued via the handlers.

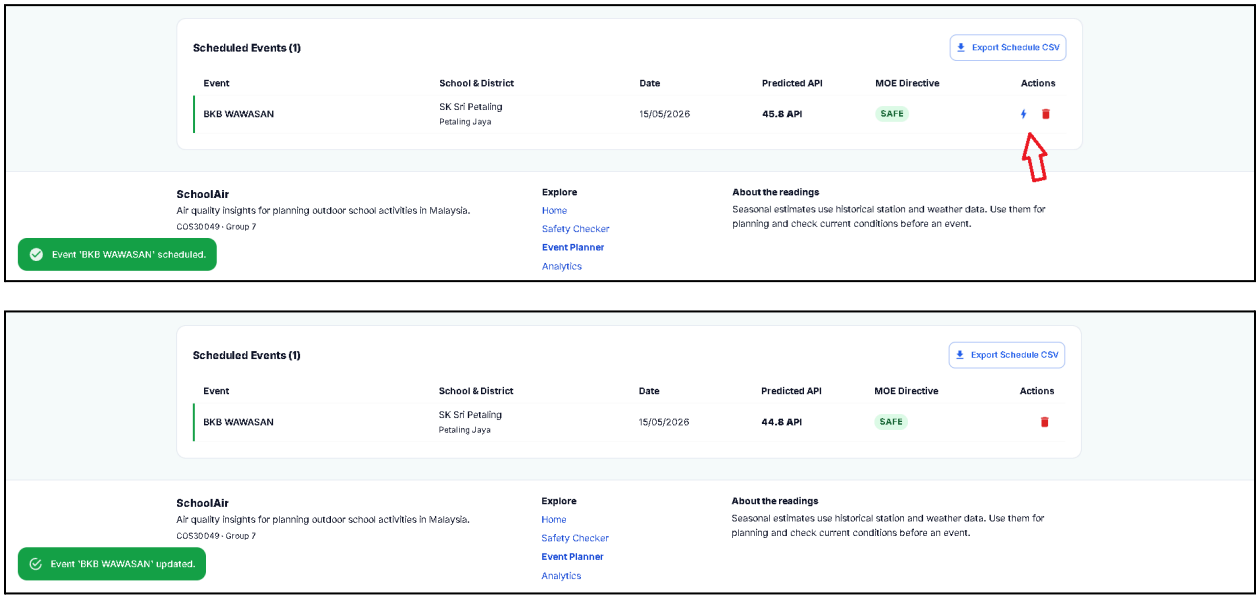


Figure 3.1 Planner demonstration for Petaling Jaya on 15 May 2026. The saved assessment was 45.8 API before the update and 44.8 API after it.

4.4 Deployment Limitations

For a group demonstration with a minor amount of changes, the local persistence model itself would be adequate. A production implementation would need a verified account or school perimeter and storage that allows concurrent changes to occur. The frontend confirmation experience and the server's handling of a stale identifier should both be tested before using the schedule in a shared environment.

5.0 API Documentation

5.1 Request Conventions and Endpoint Inventory

The development backend is available at http://localhost:8000 when launched with the repository's default Uvicorn setup. The interface calls JSON endpoints under /api/v1 and /docs provides the development interface for trying them. Read operations use GET query parameters, create and model requests use POST bodies. Event changes use PUT and DELETE with the event identifier in the path.

Table 4.1 SchoolAir Endpoint Inventory

Method and route relative to /api/v1	**Responsibility**
GET /health	Reports service status, station and forecast record counts, and model benchmark metrics.
GET /stations	Station and school directory information
GET /schools	Filter school directory
GET /forecast-lookup	Assess a station, month and event window
POST /assess-event	Assessment using a JSON request
POST /model-prediction	Run the regressor against stored seasonal features
GET /seasonal-planner	Retrieve monthly summaries and recommendations
GET /compare-districts	Compare locations at a common month and start hour
GET /analytics	Retrieve a heatmap, historical episode and metadata
GET /events	List stored events
POST /events	Create a scheduled event
PUT /events/{event_id}	Update a scheduled event
DELETE /events/{event_id}	Delete a scheduled event

The following sections provide worked examples for assessment lookup, model prediction, Planner, comparison, Analytics and schedule operations. The inventory lists the remaining exposed operations. Complete schemas and field definitions are available through the running

service's OpenAPI document. For readability, the JSON blocks show selected response fields while preserving the values observed during the verification run.

5.2 GET Forecast Lookup

The assessment endpoint executes the basic workflow of the Safety Checker. Frontend provides event date and time, district, activity type, cohort, and weather regime. With a given event date, the particular month is used for selection. Without a date given, the month parameter is valid.

Table 4.2 Forecast Lookup Inputs

Field	**Type and behaviour**
district	string; defaults to Petaling Jaya
event_date	optional string; calendar date in YYYY-MM-DD form
month	integer 1–12; defaults to April when date is omitted
start_hour	integer 0–23; default 8
duration_hours	integer 1–12; default 2
cohort and weather_regime	normalized strings; browser sends general and typical

```
GET
/api/v1/forecast-lookup?district=Petaling%20Jaya&event_date=2026-04-15&start_hour=8&duration_hours=2
```

HTTP 200 response excerpt

```
{
  "status": "success",
  "event_context": {"start_time_label": "08:00", "end_time_label": "10:00"},
  "prediction": {"predicted_api": 36.05, "peak_window_api": 37.19},
  "moe_safety": {"tier_label": "SAFE"}
}
```

The response includes the station, event context, prediction values and assessment tier. It also provides a 24-hour series and attribution data for the frontend. The underlying hourly table contains typical and peak haze estimates, allowing the chart to display scenario information while the standard assessment uses the typical series. The peak window value in the example response refers only to the selected event hours.

With start_hour set to 25, the same request returned HTTP 422 with a field-level detail. This demonstrates validation independent of the frontend. An unavailable seasonal slice has a separate 404 path.

HTTP 422 response excerpt


```
{
  "status": "validation_error",
  "message": "Input validation failed against MAAQS and system bounds.",
  "details": ["start_hour: Input should be less than or equal to 23"]
}
```


(a)



(b)

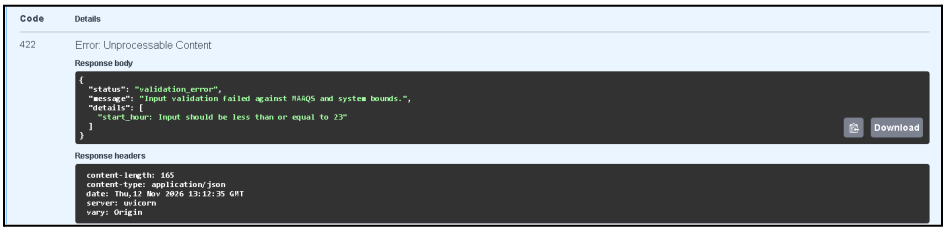


Figure 4.1 FastAPI forecast lookup for Petaling Jaya on 15 May 2026. (a) The request with start_hour=8 returns HTTP 200 and an estimate of 45.81 API. (b) The request with start_hour=25 returns HTTP 422.

5.3 POST Model Prediction

The endpoint employs XGBoost in the specified event period. It uses the assessment input model but has a few limitations as it requires standard historical features and will not accept custom modifications. The prediction is presented in a different way in the web browser to ensure that the user can tell it apart from the pre-calculated assessment.

```
POST /api/v1/model-prediction

{
  "district": "Petaling Jaya",
  "event_date": "2026-04-15",
  "start_hour": 8,
  "duration_hours": 2
}
```

HTTP 200 response excerpt

```
{"status": "success", "predicted_api": 36.0, "dataset_api": 36.05}
```

The response features a new predicted_api value and a dataset_api for checking against each other, in addition to the resolved station, event context, and resulting advice. There will be small differences related to the fact that the inputs in the underlying feature table have been rounded. The example demonstrates that the route executed and produced a comparable estimate. This one example doesn't mean that all routes have the same output for the specific stations or that any of them corresponds to the requirements for that specific date of the event.

An unavailable regressor returns HTTP 503, and an unsupported scenario returns HTTP 422. These outcomes concern the model request itself. The lookup route can continue to serve stored estimates when the model cannot run. The backend regression tests check this separation.

5.4 GET Seasonal Planner

The Planner requests a district, calendar year, cohort and weather regime. The server retrieves 12 months of summary data for the selected station. It provides a ranking for each value that was determined and puts all the months in the order necessary for the recommendations. The response includes an optimal month, three recommended months, two higher-risk months, counts by tier and all monthly records.

```
GET /api/v1/seasonal-planner?district=Petaling%20Jaya&year=2026
```

HTTP 200 response excerpt

```
{
  "status": "success",
  "district": "Petaling Jaya",
  "year": 2026,
  "recommended_months": ["April", "February", "March"],
  "high_risk_months": ["August", "September"]
}
```

The monthly data reveal both standard forecasting and highest haze analysis. The planning risk index is a different pre-existing field used for making default comparisons. The response includes forecast_basis so a consumer can explain that these profiles recur across the planning years shown in the application. A district without any matching rows returns HTTP 404.

5.5 GET District Comparison

A comparison request includes district_a, district_b, month, start_hour and cohort. The service checks the candidates for evaluation purposes using both month and start_hour values to ensure uniformity in terms of whether the difference reported represents comparable conditions. Each district object received in response carries evaluation data and the response also contains the district of preference as well as index difference among other values.

```
GET
/api/v1/compare-districts?district_a=Petaling%20Jaya&district_b=Shah%20Alam&month=4
&start_hour=8
```

HTTP 200 response excerpt

```
{"month": 4, "hour": 8, "preferred_district": "Shah Alam", "api_difference": 0.6}
```

Because the comparison result depends on the supplied month and hour, the report should record both inputs whenever it uses a screenshot or quotes an index difference.

5.6 Event Create, Read, Update, and Delete

POST /events accepts event_title, school_name, district, event_date and start_hour, plus optional inputs including duration and participant count. The server evaluates the event and returns HTTP 201 with an event identifier. GET /events returns a count and event array. An optional corridor filter narrows the list.

```
POST /api/v1/events

{
  "event_title": "SchoolAir Evidence Example",
  "school_name": "Example School",
  "district": "Petaling Jaya",
  "event_date": "2026-04-15",
  "start_hour": 8
}
```

HTTP 201 response excerpt


```
{"status": "created", "event": {"event_id": "EVT-EXAMPLE", "start_hour": 8,
"predicted_api":
      36.05, "tier_label": "SAFE"}}
```


PUT /events/{event_id} accepts changes to supported fields such as start_hour. It returns the refreshed record. DELETE /events/{event_id} removes the matching record and returns the deleted identifier. An unknown event identifier returns HTTP 404. For this report the example identifier is masked consistently because the app generates a different value on each create.

Update handler response excerpt


```
{"status": "updated", "event": {"event_id": "EVT-EXAMPLE", "start_hour": 9,
"predicted_api":
      38.53, "tier_label": "SAFE"}}
```

Delete handler response excerpt

```
{"status": "deleted", "deleted_event_id": "EVT-EXAMPLE"}
```

The rules regarding dates and times are applied even when a request is sent to FastAPI directly. In the test suite, a create request with an impossible date returned HTTP 422 and did not call the persistence function. This matters because an invalid record could otherwise appear in future schedule loads.

5.7 GET Analytics and Station Context

The GET /analytics endpoint accepts district and corridor query parameters. The server returns the historical episode for the selected district together with a set of seasonal heatmap records. The corridor parameter further filters the heatmap rows. When all corridors are selected, the heatmap contains 312 district-month records. The selected station determines the historical episode and helps the frontend identify which station to highlight.

```
GET /api/v1/analytics?district=Petaling%20Jaya
```

HTTP 200 response excerpt

```
{
  "status": "success",
  "selected_district": "Petaling Jaya",
  "selected_corridor": "All Corridors",
  "seasonal_heatmap": [{"district": "Balik Pulau", "state": "Pulau Pinang", "corridor":
      "Northern Corridor", "month": 1, "month_short": "Jan", "pred_mean_api": 43.65,
      "pred_peak_haze_api": 67.31, "planning_risk_index": 43.7, "mean_temp": 27.83,
      "mean_humidity": 76.02}]
}
```

The corridor parameter filters the heatmap rows. When `district="All Stations"`, the backend uses the first listed station in the selected corridor for the historical episode. With All Corridors selected, the heatmap includes all supported stations and the historical episode uses Petaling Jaya.

The response also includes model metrics and correlation data, which describe the backend output. These fields should not be confused with the two visible analytics tabs, which display the heatmap and 7-day trend. The frontend uses the monthly records to determine the heatmap values and the episode series to generate the time plot. This arrangement allows both views to use a shared request while maintaining separate visual interactions.

Application startup includes station and school metadata fetched using GET /stations. A separate GET /schools directory route is available for directory queries. Public-facing implementation uses station keys obtained from these routes, meaning that new stations require a complete record in both metadata and prepared data tables. Otherwise, they might be included in a selector and find themselves without a useful historical profile.

5.8 API Coverage and Conventions

Table 4.1 lists the available API endpoints. The worked examples are those that have the closest relation to the process as experienced by the user, while the full source ZIP archive includes the code for these routes. The JSON keys in the examples use the correct spelling, and the status codes are separated from the response status. A client should handle both transport errors and the structured messages returned by the server.

6.0 Model Integration

6.1 Model Selection From Assignment 2

Assignment 2 projected the comparison between linear regression, random forest, and XGBoost for the purpose of predicting the continuous index. In the Assignment 2 benchmark, XGBoost achieved the lowest MAE and RMSE and the highest R^2 among the three models evaluated. Its MAE was 6.9553 API points, its RMSE was 9.0705 API points, and its R^2 was 0.6123 (Group 7, 2026b). Linear regression proved to be competitive, with MAE of approximately 7.06 points. Thus, the fact that the models are different should be addressed in terms of the benchmark values, not by stating the big gain in accuracy.

Table 5.1 Assignment 2 Continuous API Prediction Benchmarks

Model	**RMSE in API points**	**MAE in API points**	**R^2**
Linear Regression	9.3383	7.0588	0.5891
Random Forest Regressor	9.2917	7.0875	0.5932
XGBoost Regressor	9.0705	6.9553	0.6123

Source — Group 7 (2026b), Table 3.1

MAE is the average error magnitude on the tested dataset. RMSE takes large errors into account more than others. R^2 explains the proportion of variability in comparison with the benchmark. However, none of the mentioned statistics say anything about the probability of a forecast being correct. They aim to provide insights on the benchmark problem presented in Assignment 2, and it is necessary to apply a separate seasonal analysis for SchoolAir's monthly and hourly seasonality patterns.

The Assignment 2 classification test set contained 55 Move Indoors examples and no Total Suspension cases. The small number of high-risk cases limits what the benchmark can tell us about classifier performance in those categories (Group 7, 2026b).

6.2 Preparing Historical Profiles For Serving

The generator collects the historical records according to the location, month, and hour. It allows summing up the applied inputs, including the index and weather items. The reliance on the lagged and rolling inputs provides persistence for the case. These inputs get passed to the model in the predefined order. The generator utilizes the station selection algorithm of the second assignment. The data from the generator is characterized by certain join requirements where the district metadata, hourly index, and weather records must match because Kelantan, Terengganu, and Pahang regions do not have station details, and there are no output rows for these regions in the final outcome.

In addition to that, the generator produces the normal estimate and the extreme haze estimate for the peak haze situation with the help of different historical inputs. It is important to note that the extreme haze estimates refer to the artificial scenario. They should be considered as analysis and not as a prediction of the haze occurrence. The offline procedure enables calculating the contribution of features for the normal profile and saves the aggregated data in parallel with the hourly forecasts.

Monthly summaries enfold statistics obtained from historical observations and forecasts derived from the created hourly records. The planner's risk index is the guiding parameter of the generator's planning activity. It differs in essence from the conventional index of hourly results as well as the index value achieved in a particular event period, while temperature, humidity, and wind data are added to the context. Safe-hour percentages from historical records demonstrate summaries of the information grouped by month, while the proposed months come from effective planning data.

6.3 Feature Schema

Table 5.2 Feature Groups Used By The Packaged Model

Group	Fields	Purpose In The Input Profile
Location	Latitude, Longitude	Distinguish monitoring locations
Index and History	API, Lag_1h, Lag_24h, Rolling_Mean_6h	Represent the current profile and persistence
Weather	Temperature, humidity, wind_speed	Represent accompanying weather conditions
Time	Hour, month, day_of_week	Identify the profile's time context

With a fixed feature schema, a model call can't mistakenly interpret one variable as something else. While on-demand, the day-of-week field is populated with the same representative value used by the offline generator, in which the calendar date takes the role of indicating a month instead of producing a certain date forecast.

6.4 Loading The Model and Executing A Request

The backend tries to first load the model JSON files. If the model fails to be loaded from the JSON, it attempts to load the respective joblib asset. The feature list follows the JSON-first approach, where a joblib or built-in schema is available during the loading process. This way of packaging allows the server to begin with already prepared model files without executing training again.

The dedicated model endpoint refuses to accept unsupported inputs and checks whether the regressor is available. It triggers the common assessment logic that handles the identification of the station and event context. It also chooses the data from the stored rows for the times of interest and fills in the necessary model columns. The estimator produces the results for each row. The backend clips each hourly prediction to the 15–350 API range, averages the predictions across the event window, and rounds the new average to one decimal place. The response also includes the stored lookup estimate as a separate baseline.

Table 5.3 Verified Petaling Jaya Event Results

Measure	Observed Result	Meaning
Selected window	15 April 2026, 08:00-10:00	Two selected profile hours
Lookup prediction	36.05 API	Mean of the stored hourly predictions
On-demand prediction	36.0 API	Mean from the package regressor, rounded by the endpoint
Lookup peak in the window	37.19 API	Largest stored estimate in the event window
Cleanest eligible	08:00, 34.91 API	Lowest index from 07:00 through 18:00

Because rounding in the display conceals slight variations in values, that’s why the examples of responses show true JSON data. The common safety mapping, then, extracts a kind from the index as well as the cohort. In the user interface, the standard cohort is used in the checked

process. There are alternative cohort paths in the back end, and they ought to be established as API options unless used through observable control.

6.5 Postprocessing, Recommendations and Explainability

A common search chooses hours associated with the event. The average value and peak are calculated, the start hour is preserved for weather and attribution context, and candidate hours are chosen. The frontend rounds the main index term with the help of back-end recommendations. Rounding numbers in the browser is a way of presentation and does not impact tier limits.

Table 5.4 General Cohort Categories Implemented By SchoolAir

Index Used By The Application	**Tier Shown**
Up to and including 50	Safe
Above 50 through 100	Caution
Above 100 through 200	Move Indoors
Above 200	Total Suspension

The categories of the application are those that have been implemented. The prior report describes the policies of the project that are relevant to the school’s decision on the day, while the implementation of the seasonal recommendation provides evidence to be utilized in the planning process, which indicates the source of input and limitations in the printed report and interface.

The SHAP bars help in comprehending the outcome of the start-hour model. A signed contribution implies the direction of the variable's influence compared to the model's baseline level. An absolute share is a good rank, yet it does not have any direction by itself. The contributions signify the input that the model operated with, but they do not show that any variable has caused the change in reality.

The displayed confidence percentage comes from adjusted classifier scores stored in the prepared dataset. These scores are separate from the regression error metrics and have not been calibrated as probabilities that the event recommendation is correct.

7.0 Testing and Evaluation

7.1 Verification Approach

Application testing was done to find out if the input from the user, the processing by the server, and the display match each other. It involved running a frontend build and automating the process of testing both frontend and backend applications. The application itself was checked using different approaches and different parameters to check for possible errors. In a production build, the browser bundle is checked for its compilation, while API testing is done to observe any changes in observable requests. The interface check report is focused on ease of navigation and whether a user can reach the final goal or not.

Table 6.1 Verified Application Checks

Check	**Results**	**Evidence Scope**
Frontend tests	Four passed	Home rendering and date parsing or formatting
Production frontend build	Compiled successfully	Optimized browser bundle
Backend tests	Five passed	Validation, comparison, separate model run and unavailable-model handling
Lookup and model requests	Both returned HTTP 200	One matched Petaling Jaya event
Invalid start hour	HTTP 422	Request sent with hour 25
Event lifecycle	Create, read, update, and delete succeeded	Isolated temporary schedule; update and delete handlers called directly

Tests of the date utility involve impossible and imprecise calendar inputs. The back-end tests ensure that an impossible date does not call the storage writer. Also, another test checks that the on-demand request can call the regressor and that the expected amount of event hours is supplied to it and the pre-calculated result is available. Appendix K contains the backend regression test for on-demand prediction.

The checks were conducted on the working copy of the local project. It confirmed the expected scenarios but did not provide any measurements of the capacity of the respective load or a distribution of the statistical errors. There are no throughput and latency values reported. Thus, performance information in the conclusion should refer to the implementation as well as the functionality observed and not a performance benchmark which was never conducted.

7.2 UI interactions and Responsiveness

The previous review regarding the browser went over Home, Safety Checker, Planner, and Analytics. Changes in control of charts were examined, as well as execution of models, monthly selection, comparison of districts, and navigation through the sample viewport widths. It also verified that mobile navigation is present in the phone and tablet display sizes while ensuring that the document does not get horizontal overflow. It's worth mentioning that the chart or table can provide its own scrolling area.

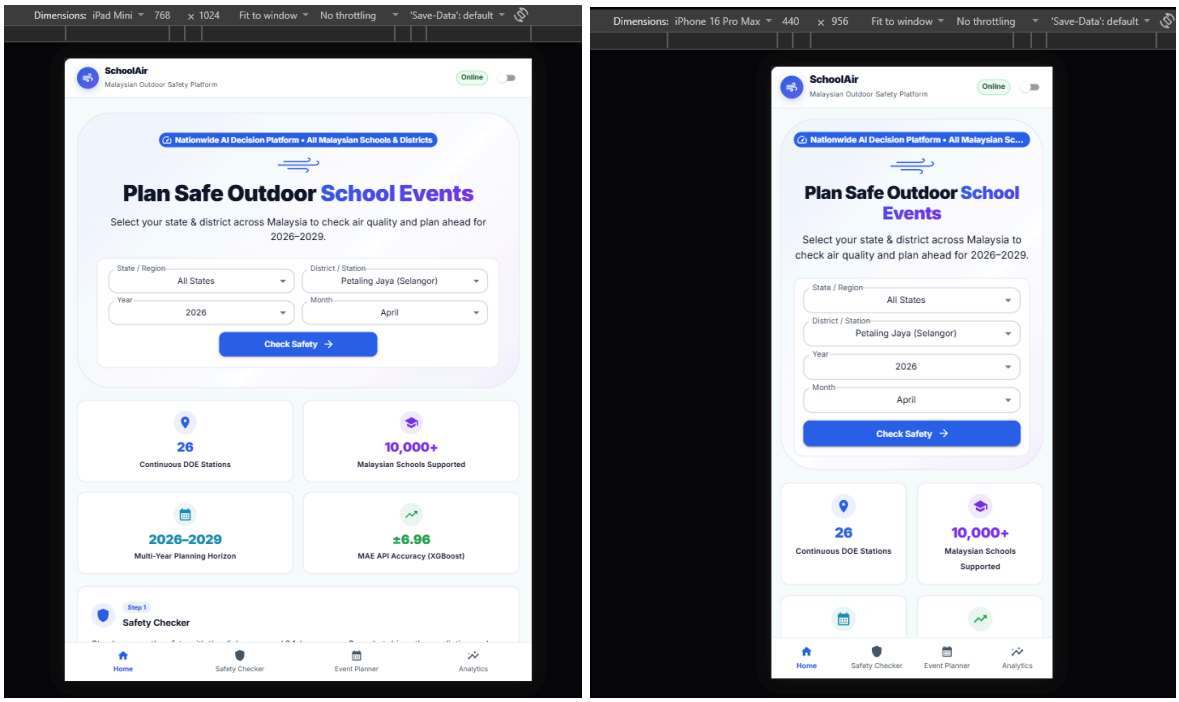


Figure 5.1 Safety Checker Across Desktop, Tablet, and Phone Widths

A responsive design should maintain its context as it transitions to different layouts. The time window and the estimate need to remain visible without depending on the placement of the card. At lower widths, the elements combine into one operation, the gauge adapts its size according to the card, and the navigation is moved to the bottom. A test for dark mode is another aspect of presentation since the marks on the chart, the axes, and hover information all require contrast with another background.

Keyboard and touch behavior needs to be tested separately from the effects of mouse hovering. When using the keyboard, it is necessary to be able to inspect or select any point. In the case of

touch, it is necessary to have a clear view of the result without the use of the cursor. The confirmation of this was done using the browser inspector of representative controls while achieving full accessibility would require more thorough testing of the keyboard, screen reader, and devices.

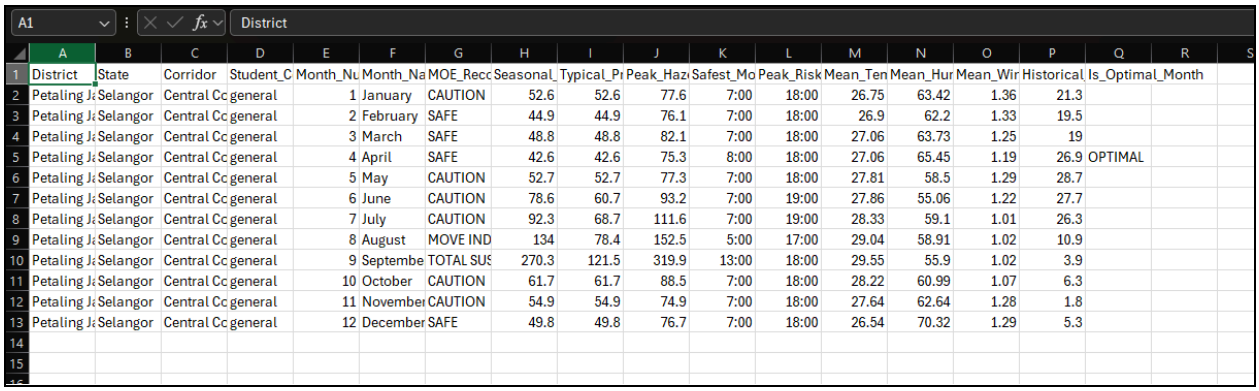
7.3 CSV and Print Verification

The Safety CSV contains the row for every of the 24 hours. It has the event metadata, the hour indicators within the considered window, average and maximum haze estimates, weather figures and summary fields. This CSV is helpful in ensuring that the considered window is visible in the raw export. The Planner CSV has 12 rows of the months of the year, which allows determining the seasonal variance and comparing the months.

During verification, 3 columns of the Planner CSV were blank at the beginning, because the export was expecting some fields that were not sent by the endpoint. The backend was modified to return the maximum risk hour, average wind speed and the percentage of hours considered safe based on the monthly data, and the verification of the export showed that those fields were filled for all 26 stations involved. This was an issue about the data contract between the browser and the server, since the values were present in the dataset, while the only issue was that they were not exposed via API response.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
1	School_Na	Matched_I	Corridor	Event_Dat	Month	Activity_Ty	Student_C	Hour_24H	Is_Selecte	Predicted_	Predicted_	Predicted_	MOE_Safe	Temperati	Humidity_	Wind_Spe	Overall_Ev	Model_Co	Dominant	Inference_Source			
2	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	0:00	NO	39.6	39.6	74.2	SAFE	24.2	74.1	0.73	36	95.3	Current St	Pre-Computed XGBoost Dataset			
3	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	1:00	NO	37.9	37.9	73.1	SAFE	23.8	74.9	0.72	36	95.3	Current St	Pre-Computed XGBoost Dataset			
4	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	2:00	NO	38.5	38.5	71.8	SAFE	23.5	75.7	0.71	36	95.3	Current St	Pre-Computed XGBoost Dataset			
5	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	3:00	NO	37.4	37.4	72.4	SAFE	23.3	76.3	0.71	36	95.3	Current St	Pre-Computed XGBoost Dataset			
6	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	4:00	NO	37.8	37.8	71	SAFE	23.1	76.7	0.72	36	95.3	Current St	Pre-Computed XGBoost Dataset			
7	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	5:00	NO	37.4	37.4	70.2	SAFE	22.9	77	0.71	36	95.3	Current St	Pre-Computed XGBoost Dataset			
8	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	6:00	NO	36.8	36.8	70.2	SAFE	23.2	76.5	0.7	36	95.3	Current St	Pre-Computed XGBoost Dataset			
9	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	7:00	NO	35.1	35.1	69.3	SAFE	24	74.8	1.52	36	95.3	Current St	Pre-Computed XGBoost Dataset			
10	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	8:00	YES	34.9	34.9	69.4	SAFE	25.4	71.8	1.61	36	95.3	Current St	Pre-Computed XGBoost Dataset			
11	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	9:00	YES	37.2	37.2	70.7	SAFE	27.2	65.2	1.71	36	95.3	Current St	Pre-Computed XGBoost Dataset			
12	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	10:00	NO	39.9	39.9	73.7	SAFE	28.7	58.5	1.86	36	95.3	Current St	Pre-Computed XGBoost Dataset			
13	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	11:00	NO	41.5	41.5	77	SAFE	30.4	53.7	1.96	36	95.3	Current St	Pre-Computed XGBoost Dataset			
14	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	12:00	NO	45.6	45.6	79.2	SAFE	31.9	50	2.07	36	95.3	Current St	Pre-Computed XGBoost Dataset			
15	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	13:00	NO	46.7	46.7	79.2	SAFE	32.9	47.4	2.07	36	95.3	Current St	Pre-Computed XGBoost Dataset			
16	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	14:00	NO	47.7	47.7	77.2	SAFE	33.2	46.5	2.05	36	95.3	Current St	Pre-Computed XGBoost Dataset			
17	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	15:00	NO	48	48	77.5	SAFE	32.7	48	1.95	36	95.3	Current St	Pre-Computed XGBoost Dataset			
18	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	16:00	NO	49.1	49.1	78.1	SAFE	31.6	52.1	1.06	36	95.3	Current St	Pre-Computed XGBoost Dataset			
19	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	17:00	NO	50.1	50.1	81.3	CAUTION	30.2	57.5	0.94	36	95.3	Current St	Pre-Computed XGBoost Dataset			
20	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	18:00	NO	51.5	51.5	81.1	CAUTION	28.8	62.3	0.86	36	95.3	Current St	Pre-Computed XGBoost Dataset			
21	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	19:00	NO	50.5	50.5	80.8	CAUTION	27.3	66.3	0.79	36	95.3	Current St	Pre-Computed XGBoost Dataset			
22	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	20:00	NO	48.6	48.6	77.9	SAFE	26.2	69	0.77	36	95.3	Current St	Pre-Computed XGBoost Dataset			
23	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	21:00	NO	46.9	46.9	77.8	SAFE	25.5	70.8	0.75	36	95.3	Current St	Pre-Computed XGBoost Dataset			
24	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	22:00	NO	42.3	42.3	78	SAFE	25	72.1	0.77	36	95.3	Current St	Pre-Computed XGBoost Dataset			
25	SK Sri Peta	Petaling Ja	Central Cc	15/04/202	April	Annual Sp	Cohort A:	23:00	NO	40.7	40.7	76.5	SAFE	24.5	73.3	0.73	36	95.3	Current St	Pre-Computed XGBoost Dataset			
26																							

Figure 5.2 Safety CSV Export Samples



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	District	State	Corridor	Student_C	Month_Nu	Month_Na	MOE_Recc	Seasonal_	Typical_Pr	Peak_Haz	Safest_Mo	Peak_Risk	Mean_Ten	Mean_Hur	Mean_Wir	Historical	Is_Optimal_Month	
2	Petaling Ja	Selangor	Central Cc	general	1	January	CAUTION	52.6	52.6	77.6	7:00	18:00	26.75	63.42	1.36	21.3		
3	Petaling Ja	Selangor	Central Cc	general	2	February	SAFE	44.9	44.9	76.1	7:00	18:00	26.9	62.2	1.33	19.5		
4	Petaling Ja	Selangor	Central Cc	general	3	March	SAFE	48.8	48.8	82.1	7:00	18:00	27.06	63.73	1.25	19		
5	Petaling Ja	Selangor	Central Cc	general	4	April	SAFE	42.6	42.6	75.3	8:00	18:00	27.06	65.45	1.19	26.9	OPTIMAL	
6	Petaling Ja	Selangor	Central Cc	general	5	May	CAUTION	52.7	52.7	77.3	7:00	18:00	27.81	58.5	1.29	28.7		
7	Petaling Ja	Selangor	Central Cc	general	6	June	CAUTION	78.6	60.7	93.2	7:00	19:00	27.86	55.06	1.22	27.7		
8	Petaling Ja	Selangor	Central Cc	general	7	July	CAUTION	92.3	68.7	111.6	7:00	19:00	28.33	59.1	1.01	26.3		
9	Petaling Ja	Selangor	Central Cc	general	8	August	MOVE IND	134	78.4	152.5	5:00	17:00	29.04	58.91	1.02	10.9		
10	Petaling Ja	Selangor	Central Cc	general	9	Septembe	TOTAL SUS	270.3	121.5	319.9	13:00	18:00	29.55	55.9	1.02	3.9		
11	Petaling Ja	Selangor	Central Cc	general	10	October	CAUTION	61.7	61.7	88.5	7:00	18:00	28.22	60.99	1.07	6.3		
12	Petaling Ja	Selangor	Central Cc	general	11	Novembe	CAUTION	54.9	54.9	74.9	7:00	18:00	27.64	62.64	1.28	1.8		
13	Petaling Ja	Selangor	Central Cc	general	12	December	SAFE	49.8	49.8	76.7	7:00	18:00	26.54	70.32	1.29	5.3		
14																		
15																		

Figure 5.3 Planner CSV Export Samples

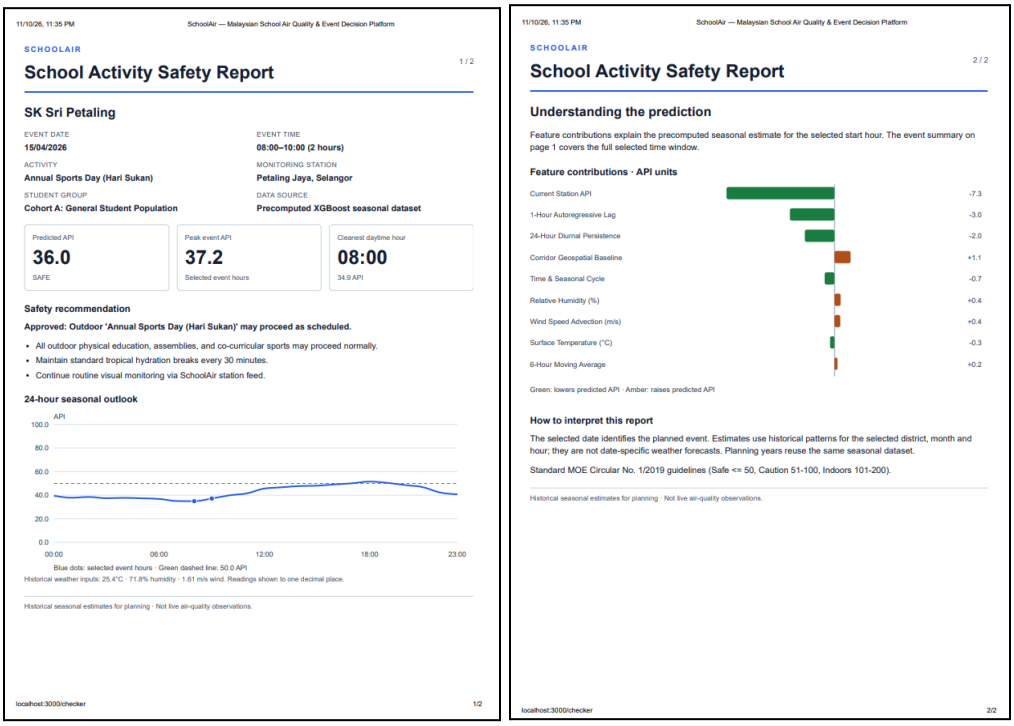


Figure 5.4 Printed SchoolAir Assessment

Different start hours were used in the provided CSV and printed example. The figures were thus compared by event context before testing for agreement. A new comparison to the report should be based on both files using the same station, event date, and start time.

7.4 Evidence For Advanced Functionality

To validate the advanced functionality requirements of the rubric, we must apply tasks that the users perform and record the outputs of those tasks. The on-demand model request example makes use of the packaged model regressor to perform a modeled response to the user-selected event. The campus comparison uses model-based estimates to compare air quality at two locations for the same selected month and start hour. The scheduled event includes a valued evaluation that can be made permanent through the event recording, while exports allow the user to see those records outside of the browser.

Table 6.2 Advanced Workflow Evidence

Workflow	**User Action**	**Observable Outcome**
On-demand prediction	Select an event, then click Run Model Prediction.	Separate recomputed value and preserved dataset assessment
Campus comparison	Choose two districts at a shared month and hour	Two estimates, difference and preferred location
Schedule update	Change a saved event start hour	Recalculated stored result for the same event identifier
Export and print	Download CSV or open the printable report	Portable evidence with selected event context

The four D3 chart models consist of a gauge indicator, a line graph, a horizontal SHAP chart and a colormap. The two types of line diagrams differ in interaction features although they belong to the same type of chart. The assessment format line helps to pick the hour, while the episode format one works with history using a brush. The consideration of both formats separately as two distinct types would be diluting the part that gauge, bar chart and heat map play.

The screenshots show how a user moves from selecting an event on the Home page to checking its estimated air quality, running the model on demand, and exporting the assessment.

7.5 Assignment 1 & 2 Risk Resolution and Final Handover Matrix

To ensure project traceability from the Assignment 1 Project Management Plan and the Assignment 2 ML Modelling Report, the table below presents the final resolution of all Phase 3 operational and technical risks (previously reported as Risks 07 to 11).

Table 6.3 Final Phase 3 Risk Monitoring and Resolution Matrix

Risk ID & Scope	**Phase 3 Status**	**Phase 3 Final Resolution & Verification**
Risk 07 (CORS & Schema)	Closed	Configured CORSMiddleware(allow_origins=["*"]); Pydantic v2 schemas aligned with Axios across all 13 endpoints.
Risk 08 (Input Boundaries)	Closed	Client regex & calendar validation paired with server Pydantic bounds (returns HTTP 422 on invalid hours like 25).
Risk 09 (D3.js Rendering)	Closed	The chart cleanup stops active D3 transitions and removes old SVG elements when the chart component unmounts.
Risk 10 (Model Serialization)	Closed	The backend loads the packaged XGBoost model from champion_regressor.json.
Risk 11 (Team Contribution)	Closed	The contribution form records each member's responsibilities and completed work.

8.0 Conclusion and Future Work

8.1 Project Outcomes

In this application, SchoolAir implements the planning workflow described in Assignment 1 and integrates the XGBoost model selected in Assignment 2. This fully functional software utilizes the specified event to extract and process the data through FastAPI, and conveys the outcome via React. D3 Visualization allows one to analyze the event mean, variations by hour, contributions of model features, and seasonal patterns in the regions.

The implemented example from Petaling Jaya gives evidence of the practical importance of the solution. The user may use a two-hour event approximation, comparing it with an hour that shows the best values, alongside the hourly curve of events and calculations conducted with the same historical feature that has been preserved. The Planner enables the user to make the place comparison and a monthly overview.

Each assessment identifies the station, month, and event hours used for the estimate. The on-demand model run uses historical features for the same selection, so users can compare it with the stored result while keeping the event context visible. Both estimates support seasonal planning.

8.2 Priorities For Future Development

Coverage in geography has become one of the highest aims for the widening scope of operations. This section will describe the process of including the states of Kelantan, Terengganu, and Pahang with a thorough analysis of the data and model presented prior to the implementation of the new locations. The next priorities of this kind include seasonal evaluation with sufficient occurrence of high-risk instances, specially labeled feed with the current observations, as well as enhanced control over accounts and storage used in schools.

8.3 Expanding Coverage In Peninsular Malaysia

Kelantan, Terengganu and Pahang take priority due to the lack of existing monitoring stations from this application in those areas. For assignment 3, the same corridor and station selection from assignment 2 is reused. Such a lack of coverage shows one of the limitations of packaging project information. It is not clear whether any monitoring stations or records about environmental issues exist outside the selected data.

The first step is to identify candidates for monitoring stations and gather historical data about Air Pollutant Index and weather for the same locations and periods. Every record must have valid station identification, validated location, and time measurements. It is required to understand missingness by month and hour, identify duplicates in records, and include gaps in records.

The next stage is to assess the existing regressor at the introduced locations using more recent observations and certain data sets on which no predictions were made. Assess performance by type of station, time of year, and risk category. It is important to check rare cases because the average errors can mask certain deficiencies in performance. If the model cannot be successfully used in new locations, it is possible to train it again or adjust its parameter values using regional data and re-examine the storage locations for data not seen by the model.

After a station has passed the data check and the regressor check, the station profile has to be checked to generate the hourly and monthly station profile and the historical episodes gathered. With this in mind, it is necessary to check that coverage matches the resulting state choices.

Table 7.1 Proposed Release Checks For New Stations

Release Check	**Evidence Required**
Data readiness	Verified station metadata and sufficient matched hourly observations
Model readiness	Held-out results by station, season and risk tier
Application readiness	Consistent station identity and values across Checker, Planner and exports
Unsupported locations	Clear message and no unrelated station substituted

9.0 Bibliography

1. FastAPI (n.d.) FastAPI documentation. Available at: https://fastapi.tiangolo.com/.
2. Group 7 (2026a) SchoolAir project planning report. COS30049 Assignment 1. Unpublished group report.
3. Group 7 (2026b) SchoolAir: ML Modelling Report. COS30049 Assignment 2. Unpublished group report.
4. Group 7 (2026c) SchoolAir application and prepared datasets. COS30049 Assignment 3. Local repository working copy and verification outputs.
5. Observable (n.d.) What is D3? Available at: https://d3js.org/what-is-d3.
6. React (n.d.) Quick Start. Available at: https://react.dev/learn.
7. Swinburne University of Technology (2026) COS30049 Assignment 3 brief and marking rubric. Canvas course material.

Generative AI use. We used ChatGPT to draft reports, check technical claims against the project files, and troubleshoot and verify the source-code package.

10.0 Appendices

Appendix A: Client Assessment Request (App.js)

The browser submits the event selection and accepts a response only when it belongs to the active request.

frontend/src/App.js, lines 503–542

```
const fetchEventAssessment = useCallback(async () => {
    const requestId = ++assessmentRequest.current;
    setLoading(true);
    setAssessmentError('');
    if (!/^\d{4}-\d{2}-\d{2}$/.test(eventDate)) {
    setAssessmentError('Enter a valid event date.');
    setLoading(false);
    return;
    }
    try {
    const params = {
    district: selectedDistrict,
    custom_school_name: customSchoolName,
    event_date: eventDate,
    month: selectedMonth,
    start_hour: startHour,
    duration_hours: durationHours,
    activity_type: activityType,
    cohort,
    weather_regime: weatherRegime,
    };
    const res = await axios.get(`${API_BASE}/forecast-lookup`, { params });
    if (requestId !== assessmentRequest.current) return;
    setAssessment(res.data);
```

```
      setLoadedAssessmentKey(assessmentSelectionKey);
        setBackendOnline(true);
        } catch (err) {
        if (requestId !== assessmentRequest.current) return;
      setBackendOnline(Boolean(err.response));
      setAssessmentError(err.response?.data?.details?.join(' ') || err.response?.data?.message
        || 'Unable to retrieve an assessment. Check the backend connection.');
        } finally {
        if (requestId === assessmentRequest.current) setLoading(false);
        }
          }, [assessmentSelectionKey, selectedDistrict, customSchoolName, eventDate,
selectedMonth, startHour, durationHours, activityType, cohort, weatherRegime]);
```

Appendix B: Assessment and Calendar Validation (main.py)

The shared validation and the assessment input schema establish calendar, cohort, and numerical constraints. The extract ends within the schema.

backend/main.py, lines 270–310

```
class ValidatedEventFields(BaseModel):
    # Reuse semantic validation for body models and requests built by GET routes.
  @field_validator("event_date", check_fields=False)
    @classmethod
    def validate_calendar_date(cls, value):
    if value is not None:
    datetime.strptime(value, "%Y-%m-%d")
    return value

    @field_validator("cohort", check_fields=False)
    @classmethod
    def validate_event_cohort(cls, value):
    if value is not None:
    value = value.lower().strip()
    if value not in {"general", "athletic", "high-intensity", "asthmatic", "sensitive"}:
            raise ValueError("Invalid student cohort")
    return value

  @field_validator("weather_regime", check_fields=False)
    @classmethod
    def validate_event_regime(cls, value):
    if value is not None:
    value = value.lower().strip()
    if value not in {"typical", "peak_haze", "custom_override"}:
            raise ValueError("Invalid weather regime")
    return value
```

```
class EventAssessmentRequest(ValidatedEventFields):
    school_id: Optional[str] = Field(default=None)
    district: Optional[str] = Field(default="Petaling Jaya")
    custom_school_name: Optional[str] = Field(default=None, max_length=140)
    custom_lat: Optional[float] = Field(default=None, ge=0.8, le=7.5)
    custom_lon: Optional[float] = Field(default=None, ge=99.5, le=119.5)
    event_date: Optional[str] = Field(default=None)
    month: int = Field(default=4, ge=1, le=12)
    start_hour: int = Field(default=8, ge=0, le=23)
    duration_hours: int = Field(default=2, ge=1, le=12)
    activity_type: str = Field(default="Annual Sports Day", min_length=2,
max_length=100)
    cohort: str = Field(default="general")
    weather_regime: str = Field(default="typical")
```

Appendix C: Forecast Lookup Assessment Route (main.py)

The GET route builds a validated request and passes it to the shared assessment function.

backend/main.py, lines 703–728

```
@app.get("/api/v1/forecast-lookup", tags=["AI Forecast Lookup"])
 def get_forecast_lookup(
        district: str = "Petaling Jaya",
        school_id: Optional[str] = None,
        custom_school_name: Optional[str] = None,
        event_date: Optional[str] = None,
        month: int = 4,
        start_hour: int = 8,
        duration_hours: int = 2,
        activity_type: str = "Annual Sports Day",
        cohort: str = "general",
        weather_regime: str = "typical",
 ):
        req_obj = EventAssessmentRequest(
        school_id=school_id if isinstance(school_id, str) else None,
        district=district if isinstance(district, str) else "Petaling Jaya",
            custom_school_name=custom_school_name if isinstance(custom_school_name, str)
else None,
        event_date=event_date if isinstance(event_date, str) else None,
        month=month if isinstance(month, int) else 4,
        start_hour=start_hour if isinstance(start_hour, int) else 8,
        duration_hours=duration_hours if isinstance(duration_hours, int) else 2,
        activity_type=activity_type if isinstance(activity_type, str) else "Annual Sports Day",
        cohort=cohort if isinstance(cohort, str) else "general",
        weather_regime=weather_regime if isinstance(weather_regime, str) else "typical",
        )
        return compute_event_assessment(req_obj)
```

Appendix D: Window Selection and Aggregation Logic (main.py)

The server selects rows by station and month, identifies the event hours, and locates the start-hour row.

backend/main.py, lines 392–408

```
        target_month = dt_obj.month
        day_of_week = dt_obj.weekday()
        except ValueError:
        raise HTTPException(status_code=422, detail="Invalid event_date format. Expected
                YYYY-MM-DD.")

        day_slice = HOURLY_FORECASTS_DF[
        (HOURLY_FORECASTS_DF["district"] == target_district)
        & (HOURLY_FORECASTS_DF["month"] == target_month)
        ].sort_values("hour")

        if day_slice.empty:
        raise HTTPException(status_code=404, detail=f"No forecast found for {target_district}
in
                month {target_month}.")

        event_hours = [(req.start_hour + h) % 24 for h in range(req.duration_hours)]
        window_rows = day_slice[day_slice["hour"].isin(event_hours)]
        primary_row = day_slice[day_slice["hour"] == req.start_hour].iloc[0]
        api_col = "predicted_api_peak_haze" if req.weather_regime == "peak_haze" else
        "predicted_api_typical"
```

Appendix E: On-Demand Model Prediction Route (main.py)

The dedicated route resolves the event, constructs model inputs, and returns the computed average and the lookup baseline.

backend/main.py, lines 731–768

```
@app.post("/api/v1/model-prediction", tags=["AI Forecast Lookup"])
def run_model_prediction(payload: EventAssessmentRequest):
    """Run the regressor again using stored seasonal features for the event window."""
    if payload.weather_regime != "typical" or any(value is not None for value in (
    payload.override_api, payload.override_temperature,
    payload.override_humidity, payload.override_wind_speed,
    )):
    raise HTTPException(status_code=422, detail="This prediction uses typical historical
    features without overrides.")
    if REGRESSOR is None:
    raise HTTPException(status_code=503, detail="The prediction model is unavailable.
    Dataset readings remain available.")

    baseline = compute_event_assessment(payload)
    month = baseline["event_context"]["month"]
    hours = [(payload.start_hour + h) % 24 for h in range(payload.duration_hours)]
    rows = HOURLY_FORECASTS_DF[
    (HOURLY_FORECASTS_DF["district"] == baseline["station"]["district"])
    & (HOURLY_FORECASTS_DF["month"] == month)
    & HOURLY_FORECASTS_DF["hour"].isin(hours)
    ].copy()
    # Match the dataset generator's typical seasonal features and clipping bounds.
    rows["API"] = rows["mean_api"]
    rows["day_of_week"] = 2
    predictions = np.clip(REGRESSOR.predict(rows[FEATURE_COLS]), 15.0, 350.0)
    predicted_api = float(np.mean(predictions))
```

```
return {
"status": "success",
"predicted_api": round(predicted_api, 1),
"dataset_api": baseline["prediction"]["predicted_api"],
"moe_safety": evaluate_moe_directive(predicted_api, payload.cohort,
payload.activity_type),
"station": baseline["station"],
"event_context": baseline["event_context"],
"basis": "Recomputed from historical seasonal features for the selected event hours.
Stored inputs are rounded, so small differences from precomputed readings are
expected.",
}
```

Appendix F: Seasonal Planner Response Builder (main.py)

The endpoint converts monthly summaries to response objects used by the Planner and its CSV export.

backend/main.py, lines 790–816

```
    for _, row in sub.iterrows():
    m_int = int(row["month"])
    effective_api = float(row["pred_peak_haze_api"] if target_regime == "peak_haze" else
       row["planning_risk_index"])
    moe_info = evaluate_moe_directive(effective_api, target_cohort, "Outdoor School
Event")
    tier_counts[moe_info["tier_label"]] = tier_counts.get(moe_info["tier_label"], 0) + 1

    months_output.append({
    "month": m_int,
    "month_name": MONTH_NAMES[m_int],
    "month_short": MONTH_SHORT[m_int],
    "pred_mean_api": float(row["pred_mean_api"]),
       "pred_peak_haze_api": float(row["pred_peak_haze_api"]),
       "planning_risk_index": float(row["planning_risk_index"]),
    "effective_api": round(effective_api, 1),
    "mean_temp": float(row["mean_temp"]),
    "mean_humidity": float(row["mean_humidity"]),
    "mean_wind": float(row["mean_wind"]),
    "safe_pct": float(row["safe_pct"]),
    "safest_hour": int(row["safest_hour"]),
       "safest_hour_label": f"{int(row['safest_hour']):02d}:00",
       "peak_risk_hour_label": f"{int(row['peak_risk_hour']):02d}:00",
    "tier_label": moe_info["tier_label"],
    "risk_key": moe_info["risk_key"],
    "badge_color": moe_info["badge_color"],
```

```
    })

    sorted_by_clean = sorted(months_output, key=lambda m: m["effective_api"])
```

Appendix G: D3.js Historical Trend Brush Implementation (DiurnalTrendD3.js)

The D3 brush converts the selected overview interval into the displayed time range.

frontend/src/components/charts/DiurnalTrendD3.js, lines 395–420

```
        xFocus.domain([d0, d1]);
        const sub = parsedData.filter((d) => d.dt >= d0 && d.dt <= d1);
        if (sub.length > 0) {
        setBrushStats({
        start: d3.timeFormat('%d/%m/%Y %H:%M')(d0),
        end: d3.timeFormat('%d/%m/%Y %H:%M')(d1),
        hours: sub.length,
        peakActual: d3.max(sub, (d) => d.actual_api)?.toFixed(1),
        avgActual: d3.mean(sub, (d) => d.actual_api)?.toFixed(1),
        });
        }
        }
      pathActual.interrupt().attr('stroke-dasharray', null).attr('stroke-dashoffset',
        null).attr('d', lineActualFocus);
        pathPred.attr('d', linePredFocus);

xAxisFocusG.call(d3.axisBottom(xFocus).ticks(4).tickFormat(d3.timeFormat('%d/%m/%Y'))
);
        };

        const brush = d3.brushX().extent([[0, 0], [innerW, contextH]]).on('brush end', brushed);
        const brushG = context.append('g').attr('class', 'brush').call(brush);
        brushG.call(brush.move, [innerW * 0.15, innerW * 0.85]);

        pathActual.call(revealLine, reducedMotion);
        if (!reducedMotion) pathPred.attr('opacity', 0).transition().duration(400).attr('opacity',
        1);
```

```
brushG.selectAll('.overlay').style('pointer-events', 'none');
brushG.selectAll('.selection').style('pointer-events', 'all').style('cursor', 'grab');
```

Appendix H: Browser Client-Side CSV Export Utility (exportUtils.js)

The export utility serializes rows, escapes cells, and triggers a download through the browser.

frontend/src/utils/exportUtils.js, lines 10–37

```
export function downloadCsvFile(filename, rows) {
  if (!rows || rows.length === 0) return;

  const headers = Object.keys(rows[0]);
  const escapeCell = (val) => {
      if (val === null || val === undefined) return '';
      const str = String(val).replace(/"/g, '""');
      return /[,"\n]/.test(str) ? `"${str}"` : str;
  };

  const csvLines = [
      headers.join(','),
      ...rows.map((row) => headers.map((h) => escapeCell(row[h])).join(',')),
  ];

  const blob = new Blob([csvLines.join('\n')], { type: 'text/csv;charset=utf-8;' });
  const url = URL.createObjectURL(blob);
  const link = document.createElement('a');
  link.href = url;
  link.setAttribute('download', filename);
  document.body.appendChild(link);
  link.click();
  document.body.removeChild(link);
  URL.revokeObjectURL(url);
```

Appendix I: MOE Safety Directive Classification Engine (main.py)

The application maps a continuous index to the selected cohort's operational category. This excerpt ends within the mapping function.

backend/main.py, lines 192–226

```
def evaluate_moe_directive(predicted_api: float, cohort: str = "general", activity_type: str =
    "Standard PE Class") -> Dict[str, Any]:
    cohort_str = cohort if isinstance(cohort, str) else "general"
    cohort_key = cohort_str.lower().strip()

    if cohort_key in ("asthmatic", "sensitive", "cohort_c"):
    safe_limit, caution_limit, indoors_limit = 42.0, 75.0, 150.0
    cohort_label = "Cohort C: Asthmatic & Respiratory Sensitive"
    threshold_note = "Tightened thresholds (Safe <= 42, Move Indoors > 75 API) for
pulmonary
        sensitivity."
    elif cohort_key in ("athletic", "high-intensity", "high_intensity", "cohort_b"):
    safe_limit, caution_limit, indoors_limit = 46.0, 88.0, 175.0
    cohort_label = "Cohort B: High-Intensity Athletic Training"
    threshold_note = "Adjusted thresholds (Safe <= 46, Move Indoors > 88 API) for
elevated
    ventilation."
    else:
    safe_limit, caution_limit, indoors_limit = 50.0, 100.0, 200.0
    cohort_label = "Cohort A: General Student Population"
    threshold_note = "Standard MOE Circular No. 1/2019 guidelines (Safe <= 50, Caution
    51-100, Indoors 101-200)."

    if predicted_api <= safe_limit:
    tier_class, tier_label, badge_color, risk_key = 0, "SAFE", "#16a34a", "safe"
    headline = f"Approved: Outdoor '{activity_type}' may proceed as scheduled."
```

```
        advisories = [
        "All outdoor physical education, assemblies, and co-curricular sports may proceed
                normally.",
        "Maintain standard tropical hydration breaks every 30 minutes.",
        "Continue routine visual monitoring via SchoolAir station feed.",
        ]
        elif predicted_api <= caution_limit:
        tier_class, tier_label, badge_color, risk_key = 1, "CAUTION", "#ca8a04", "caution"
        headline = f"Proceed with Caution: Modify intensity and monitor sensitive students
        during '{activity_type}'."
        advisories = [
        "Permitted to proceed, but shorten high-exertion sprints and provide shaded rest
                intervals.",
        "Teachers must ensure asthmatic students have inhalers accessible at all times.",
        "Consider shifting the event to the recommended morning clean-air window (07:00 -
                10:00).",
        ]
        elif predicted_api <= indoors_limit:
        tier_class, tier_label, badge_color, risk_key = 2, "MOVE INDOORS", "#ea580c",
"move"
```

Appendix J: Scheduled Event Persistent Update Handler (main.py)

The update handler finds the stored event, applies supported changes, and recalculates its result. This excerpt ends before the persistence and return statements.

backend/main.py, lines 950–990

```
@app.put("/api/v1/events/{event_id}", tags=["School Event Schedule CRUD"])
 def update_scheduled_event(event_id: str = Path(...), payload: ScheduledEventUpdate = ...):
        events = load_scheduled_events()
        target_idx = next((i for i, e in enumerate(events) if e["event_id"] == event_id), None)
        if target_idx is None:
        raise HTTPException(status_code=404, detail=f"Event '{event_id}' not found.")

        existing = events[target_idx]
        if payload.event_title is not None: existing["event_title"] = payload.event_title.strip()
        if payload.event_date is not None:
        existing["event_date"] = payload.event_date
        existing["month"] = datetime.strptime(payload.event_date, "%Y-%m-%d").month
        if payload.start_hour is not None: existing["start_hour"] = payload.start_hour
        if payload.duration_hours is not None: existing["duration_hours"] =
payload.duration_hours
        if payload.cohort is not None: existing["cohort"] = payload.cohort
        if payload.participants_count is not None: existing["participants_count"] =
        payload.participants_count
        if payload.weather_regime is not None: existing["weather_regime"] =
payload.weather_regime
        if payload.notes is not None: existing["notes"] = payload.notes

        assessment = compute_event_assessment(
        EventAssessmentRequest(
           district=existing["district"],
           custom_school_name=existing["school_name"],
```

```
        event_date=existing["event_date"],
        month=existing["month"],
        start_hour=existing["start_hour"],
        duration_hours=existing["duration_hours"],
        activity_type=existing["activity_type"],
        cohort=existing["cohort"],
        weather_regime=existing.get("weather_regime", "typical"),
    )
    )
    existing["predicted_api"] = assessment["prediction"]["predicted_api"]
    existing["peak_window_api"] = assessment["prediction"]["peak_window_api"]
    existing["temperature"] = assessment["weather_outlook"]["temperature"]
    existing["humidity"] = assessment["weather_outlook"]["humidity"]
    existing["wind_speed"] = assessment["weather_outlook"]["wind_speed"]
    existing["tier_label"] = assessment["moe_safety"]["tier_label"]
    existing["risk_key"] = assessment["moe_safety"]["risk_key"]
    existing["safest_hour"] = assessment["time_optimization"]["safest_daytime_hour"]
    existing["updated_at"] = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

Appendix K: Automated Model Regression Test Suite (test_api.py)

The test checks the selected feature rows, actual model invocation, and preservation of the lookup result.

backend/tests/test_api.py, lines 55–73

```
    def test_separate_prediction_runs_model_and_preserves_dataset(self):
    payload = dict(district='Petaling Jaya', event_date='2026-07-15',
            start_hour=8, duration_hours=2)
    before = main.compute_event_assessment(main.EventAssessmentRequest(**payload))
    with patch.object(main.REGRESSOR, 'predict', wraps=main.REGRESSOR.predict) as
predict:
    status, body = asyncio.run(request('/api/v1/model-prediction', payload=payload))
    self.assertEqual(status, 200)
    predict.assert_called_once()
    features = predict.call_args.args[0]
      self.assertEqual(features['hour'].tolist(), [8, 9])
      self.assertTrue((features['month'] == 7).all())
    after = main.compute_event_assessment(main.EventAssessmentRequest(**payload))
    self.assertEqual(before, after)
    self.assertEqual(body['dataset_api'], before['prediction']['predicted_api'])
    self.assertAlmostEqual(body['predicted_api'], body['dataset_api'], delta=0.2)
    with patch.object(main, 'REGRESSOR', None):
    status, _ = asyncio.run(request('/api/v1/model-prediction', payload=payload))
    self.assertEqual(status, 503)
    self.assertEqual(before,
        main.compute_event_assessment(main.EventAssessmentRequest(**payload)))
```

Appendix L: Meeting Minutes

Meeting Minutes

Team name: Group 7

Meeting No: Week 10
Location:
Attending: All members attended
Apologies: -
Meeting start time: -

Matters arising from Previous minutes: **Yes** **No**

Issues raised from previous minutes	Discussions	Outcome (Resolved?)

Confirmation of minutes from last meeting: **Yes** **No**

Outcome of meeting:

Issue	Discussion in brief	Outcome	Action: Name and Timeline
Architecture & Technology Stack Decoupling	Specified component boundaries across the React frontend application, Python FastAPI microservice, and packaged data/model asset sections.	The system limits have been defined, dividing the UX/user interface part and the backend operations such as logic, storage, and deductions.	Wei Le to draft system architecture component diagrams and boundaries by Week 11.
Offline Asset Generation & Station Coverage	Evaluated packaged historical dataset coverage across 26 monitoring stations and 4 corridors. Found that Kelantan,	Approved the production of the hourly estimates that will be presented in 7,488 rows and the monthly forecast summary of 312 rows. Prepared to ensure	Boon Ping to write generator scripts to produce precomputed_hourly_forecasts.csv and precomputed_monthly_summary.csv by Week 11.

	Terengganu, and Pahang have no monitoring station data available.	the flow of experiences for states that are not being monitored.	
Frontend UI/UX Wireframing & D3 Planning	Reviewed layout requirements for Safety Checker, 12-Month Seasonal Planner, and macro Analytics report. Established key D3 visualization elements: Safety Dial Gauge and 24-hour API graph.	The preparation of wireframes and reusable containers has been completed. D3.js has been selected as the final variant for visual representation.	Ming Jun and Kennard to initialize the React application scaffold and set up theme components by Week 11.

Actions in brief:

Meeting closed at: -
Next meeting time, date and location: -

Meeting Minutes

Team name: Group 7

Meeting No: Week 11
Location:
Attending: All members attended
Apologies: -
Meeting start time:

Matters arising from Previous minutes: **Yes** **No**

Issues raised from previous minutes	Discussions	Outcome (Resolved?)

Confirmation of minutes from last meeting: **Yes** **No**

Outcome of meeting:

Issue	Discussion in brief	Outcome	Action: Name and Timeline
FastAPI Endpoint Architecture & Validation Bounds	Specified REST endpoints under /api/v1 (/forecast-lookup, /seasonal-planner, /compare-districts) and request validation using Pydantic models.	Finalized input constraints: start hour (0–23), duration (1–12), ISO calendar date parsing, and standard cohort categories.	Boon Ping to build FastAPI route handlers and Pydantic validation schemas (ValidatedEventFields) by Week 12.
Dual Prediction Architecture & Model Serialization	Discussed integrating the trained XGBoost model via POST /api/v1/model-prediction alongside the fast tabular lookup route GET /api/v1/forecast-lookup.	Decided on dual-path design: lookup route uses precomputed tables, while Advanced Tools runs live inference using packaged JSON/joblib model files.	Kennard to serialize champion XGBoost regressor into champion_regressor.json and configure backend loading logic by Week 12.

Safety Checker UI & D3 Dial Gauge Construction	Reviewed frontend progress on event form controls (district, school, date, start hour, duration, activity) and D3 gauge implementation.	Approved D3 gauge design with dynamic needle transition, color-coded MOE tiers, and numeric text readouts.	Ming Jun to finalize Safety Checker form layout and integrate D3 gauge component by Week 12.

Actions in brief:

Meeting closed at:
Next meeting time, date and location: -

Meeting Minutes

Team name: Group 7

Meeting No: Week 12
Location:
Attending: All members attended
Apologies: -
Meeting start time:

Matters arising from Previous minutes: **Yes** **No**

Issues raised from previous minutes	Discussions	Outcome (Resolved?)

Confirmation of minutes from last meeting: **Yes** **No**

Outcome of meeting:

Issue	Discussion in brief	Outcome	Action: Name and Timeline
Event Schedule Persistence (CRUD Operations)	Designed persistent storage for scheduled events using server-side scheduled_events.json. Formulated routes for GET, POST, PUT, and DELETE.	Implemented CRUD endpoints with auto-recalculation of air quality tier and safest hour upon event update.	Boon Ping to implement /events CRUD route handlers in main.py by Week 13.
D3.js 24-Hour Diurnal Curve & SHAP Attribution Chart	Reviewed interactive visualization widgets: 24-hour diurnal line chart with temperature overlay and horizontal SHAP feature contribution bar chart.	Approved click-to-select hour on diurnal curve, cleanest-hour markers, and relative vs. signed SHAP attribution view.	Ming Jun to complete DiurnalTrendD3.js and connect SHAP contribution payload from backend by Week 13.

12-Month Regional Heatmap & 7-Day Haze Trend (Analytics)	Structured macro-analytics module: 12-month station heatmap and brushable D3 line chart for historical September 2019 haze episode.	Retired redundant correlation matrix. Approved 12-month heatmap corridor toggling and d3.brushX historical zoom tool.	Wei Le to prepare precomputed_multiday_episodes.csv and coordinate Analytics module integration by Week 13.

Actions in brief:

Meeting closed at:
Next meeting time, date and location: -

Meeting Minutes

Team name: Group 7

Meeting No: Week 13
Location:
Attending: All members attended
Apologies: -
Meeting start time:

Matters arising from Previous minutes: **Yes** **No**

Issues raised from previous minutes	Discussions	Outcome (Resolved?)

Confirmation of minutes from last meeting: **Yes** **No**

Outcome of meeting:

Issue	Discussion in brief	Outcome	Action: Name and Timeline
Data Export & Print Feature Audit	Evaluated CSV download functions (exportUtils.js) and CSS print stylesheet for formal report generation. Discovered 3 empty columns in Planner CSV export.	Fixed data contract: updated backend to supply safest_hour, mean_wind, and safe_pct in monthly payload. CSV and print PDF now match.	Boon Ping and Tee Ming Jun to verify CSV exports across all 26 stations by Week 14.
System Testing & Validation Suite Execution	Formulated automated test suite covering frontend component/date logic and backend HTTP API status codes.	Executed 4 frontend unit tests and 5 backend regression tests (test_api.py). Verified HTTP 422 for invalid hour 25 and HTTP 503 for missing model.	Kennard to document test suite outputs and write Appendix A source code listings by Week 14.

Campus Air Comparison & Guidance Alignment	Verified inter-district comparison logic (GET /compare-districts) between Petaling Jaya and Shah Alam (0.6 API difference in April at 08:00).	Ensured that MOE circular thresholds (Safe ≤ 50, Caution 51–100, Move Indoors 101–200, Total Suspension > 200) are accurately applied across both general and sensitive cohorts.	Wei Le to review policy wording in recommendations and finalize Section 6 text by Week 14.

Actions in brief:

Meeting closed at:
Next meeting time, date and location: -

Meeting Minutes

Team name: Group 7

Meeting No: Week 14
Location:
Attending: All members attended
Apologies: -
Meeting start time:

Matters arising from Previous minutes: **Yes** **No**

Issues raised from previous minutes	Discussions	Outcome (Resolved?)

Confirmation of minutes from last meeting: **Yes** **No**

Outcome of meeting:

Issue	Discussion in brief	Outcome	Action: Name and Timeline
Phase 3 Risk Log Closure & Handover Matrix	Conducted final audit of Phase 3 risks: CORS configuration (Risk 07), input validation boundaries (Risk 08), D3 DOM unmount cleanup (Risk 09), model loading fallback (Risk 10), and workload balance (Risk 11).	Verified all mitigation steps and confirmed all Phase 3 risks are formally closed in Table 6.3.	Wei Le to finalize Risk Monitoring and Resolution Matrix in Section 7.5 by submission.
Responsive Layout & Cross-Browser Verification	Audited UI appearance and layout scaling across desktop (1280px), tablet (768px), and mobile (390px) viewports.	Confirmed responsive behavior: mobile bottom navigation bar renders smoothly, dial scales correctly, and no horizontal scroll overflow occurs.	Ming Jun to capture final verification screenshot panels by submission.

Final Technical Report Compilation & Demonstration	Compiled all 8 report sections, API schema tables, figures, bibliography, and Appendices A through M. Reviewed video demonstration recording.	Final technical report approved by all team members. Contribution forms and meeting minutes attached. Packaging complete for Canvas submission.	All members to conduct final proofread and submit report package by assignment deadline.

Actions in brief:

Meeting closed at:
Next meeting time, date and location: -

Appendix M: Group Assessment Contribution Form

Group Assessment Contribution Form

Course: COS30049 Computing Technology Innovation

Project/Assignment Title: Assignment 3

Group Number: Group 7

1. Group Members:

Please list down all members of the group, including yourself:

1. Name:Poh Wei Le Student ID:106627706
2. Name:Ng Boon Ping Student ID:106627829
3. Name:Tee Ming Jun Student ID:106627816
4. Name:Kennard Chan Kai Lik Student ID:106632645

2.Team Task & Contribution Log

To be completed and signed by all group members. This section documents the specific tasks each member was responsible for and provides evidence of their work. All members must reach a consensus on the described contributions

- Example: " Design: The user login and registration pages using Figma."
- Example: " Documentation: The API endpoints for user authentication."
- Example: " Set up the project repository on GitHub. "

Group Member	Tasks
POH WEI LE	Project coordination, architecture boundary definition, multi-day episode data structuring, Phase 3 risk matrix resolution, and final technical report compilation.
NG BOON PING	Precomputed seasonal dataset preparation (hourly & monthly), FastAPI route engineering, Pydantic request validation, persistent CRUD schedule implementation, and CSV export data contracts.
KENNARD CHAN KAI LIK	XGBoost model packaging and deserialization, on-demand prediction route development, automated regression testing (test_api.py), and model performance evaluation documentation.
TEE MING JUN	React frontend architecture, Material UI layout design, D3.js chart implementations (Safety Dial, Diurnal Curve, SHAP bars, Regional Heatmap, Haze Trend brush), and export/print utilities.

3. Team Member Contributions:

As a group, discuss and evaluate the contributions of each team member in the table below. The final evaluations **must represent a consensus** agreed upon by the entire team.

Group Member	Contribution (100% in total)	Quality of Work (1-5)	Timeliness (1-5)	Collaboration (1-5)
POH WEI LE	25	5	5	5
NG BOON PING	25	5	5	5
KENNARD CHAN KAI LIK	25	5	5	5
TEE MING JUN	25	5	5	5

Contribution and Grading Policy:

1. **Normal Grade:** Members contributing 33% or more will receive the full project mark.
2. **Penalty:** Members contributing less than 33% will incur a penalty. Their grade will be calculated as follows:
    - (Contribution Percentage/33)×Group Mark

      Example: If the total group mark is 15, and a member contributes 20%, their grade will be (20/33)×15 = 9

POH WEI LE Signature: *WeiLe*

NG BOON PING Signature: *BoonPing*

KENNARD CHAN KAI LIK Signature: *Kennard*

TEE MING JUN Signature: *MingJun*

Date: 13 November 2026